

COMP – 385 AI CAPSTONE PROJECT

SECTION: 001

FINAL REPORT

KiddoLand: Child-Safe AI Engine for Adaptive Literacy Content

From:

GROUP 03

DECLARATION

We, Group 03: KiddoLand, confirm that this work submitted for assessment is our own and is expressed in our own words. Any uses made within it of the works of any other author, in any form (ideas, equations, figures, texts, tables, programs), are properly acknowledged at the point of use. A list of the references used is included.

Name	Student Id	Student Signature	Program Name
Maahi Patel	301420327	Maahi	Program Name: Software Engineering Technology – Artificial Intelligence
Yugundhar Pampari	301486761	Yugundhar	
Prem Poriya	301480031	Prem	
Jaturaput Jongsubcharoen	301391425	Jaturaput	Course Code: COMP 385 Course Name: AI Capstone Project Section: 001
Juan David Barrero Guerrero	301352353	Juan	

Date of Submission: 19th April 2026

Supervisor: Dr. Vinay Vaithilingam

Executive Summary

Children aged 1–10 face a critical and largely unaddressed gap in digital literacy tools: existing AI-powered platforms produce generic, one-size-fits-all content that does not adapt to a child's developmental stage, does not apply systematic safety screening to AI outputs, and cannot be used in institutional settings such as schools and libraries due to privacy regulations (COPPA, PIPEDA, GDPR) that prohibit collecting personal child data. The result is that parents must spend hours manually finding or writing age-appropriate content, while teachers must simply avoid AI tools altogether.

KiddoLand directly eliminates this gap by providing parents, teachers, and librarians with an AI-powered platform that generates personalized stories, rhymes, and interactive educational quizzes that are age-appropriate, safety-screened, and privacy-compliant—reducing the time to create engaging educational content from hours to under two minutes.

We solved this problem by building KiddoLand as a full-stack web application using a Python FastAPI backend with a custom HMAC-SHA256 authentication system, the Hugging Face Inference API for LLM inference and TTS narration, a React/TypeScript frontend with Material UI for accessibility and usability, MongoDB for optional data persistence, and the all-MiniLM-L6-v2 sentence-transformer model for semantic book recommendations ranked by cosine similarity.

Our key findings confirm the technical soundness of the approach: the safety filter achieved a 94% first-pass compliance rate across 50 tested inputs; child-name extraction from free-form prompts reached 92% accuracy; story and rhyme generation completes in 2–5 seconds of LLM inference; rhyme pattern adherence reached 87–91% across tested styles; and semantic book recommendations delivered 100% on-topic results in the top 5 of a 50-candidate pool—outperforming keyword-only search by 40 percentage points on our tested queries.

What distinguishes KiddoLand from platforms like AI Dungeon or generic ChatGPT wrappers is our constraint-first architecture: safety validation and age-appropriate constraints are applied both before the LLM is called and again on the generated output—never trusting the model's fine-tuning alone. This is combined with a dual-mode design that serves both personalized home use (with an account, history, and favorites) and fully anonymous institutional sessions on the same codebase, without any architectural duplication.

Based on our results, we recommend extending the project with real vision AI to replace the current filename-based image captioning, BLEU/ROUGE-based automated content quality scoring, native iOS/Android applications, and a teacher analytics dashboard for aggregated, anonymized classroom session data.

Table of Contents

Executive Summary	2
Table of Contents	3
List of Illustrations	6
Figures	6
Tables	6
1. Introduction.....	7
1.1 Background to the Problem	7
1.2 Problem Statement	7
1.3 Significance of the Problem	8
1.4 Summary of Literature Review.....	8
1.5 Summary of Exploratory Data Analysis	9
2. Methodology	10
2.1 Phase 1 Planning, Architecture, and Requirements (Weeks 1–3).....	10
2.2 Phase 2 Backend Core: Authentication, Safety, and Story Generation (Weeks 4–6)	11
2.3 Phase 3 Frontend Core: Design System and Story Page (Weeks 7–9)	11
2.4 Phase 4 Rhyme Generation Engine (Weeks 10–11)	12
2.5 Phase 5 Learning Activity Quiz, Book Recommendations, and Institution Mode (Weeks 12–13).....	12
2.6 Phase 6 Quality Assurance, Integration Testing, and Documentation (Week 14)	13
3. Results, Data, and Analysis.....	14
3.1 AI Generation Performance	14
3.2 Age-Guidance Effectiveness	15
3.3 Safety Filtering Analysis.....	15
3.4 Comparison with Conventional Systems.....	16
3.5 Software Metrics Analysis	16
4. Testing and Quality Assurance	18
4.1 Testing Strategy.....	18
4.2 Testing Environments.....	18
4.3 Swagger API Testing All Endpoints	18
4.4 Backend Verification Script Testing.....	23
4.5 Unit Tests All 31 Test Cases	24
4.6 Integration Tests API Endpoints (15 test cases)	26
4.7 System Testing End-to-End User Flows	27
4.8 User Acceptance Testing (UAT)	27

4.9 AI-Specific Testing	28
4.10 Data Quality and Input Validation	28
5. AI Ethics	29
5.1 Bias in Training Data	29
5.2 Data Privacy Designed Into the Architecture	29
5.3 Accountability and Responsibility	30
5.4 Security and Access Control	30
5.5 Accessibility	31
5.6 Transparency	31
5.7 Child Safety as Core Design Principle	32
6. Conclusions.....	33
7. Recommendations	34
8. Bibliography	35
Appendices	37
Appendix A Stakeholder Register	37
Appendix B System Specification	38
2.i Use Case List	38
2.ii Use Case Diagram	38
2.iii.i Detailed Use Case: UC-03 Generate Personalized Story (AI Use Case).....	40
2.iv Functional Requirements.....	41
2.v Non-Functional Requirements.....	43
Appendix 3 System Design	45
3.i Technology Stack	45
3.ii AI Capability Description.....	46
3.iii AI Pipeline Architecture (Flowchart).....	52
3. iv Full-Stack Architecture Diagram	52
3.v. Class Diagrams	53
3.vi. Component Diagram.....	55
3.vii. Interaction Diagrams for AI Use Cases	58
Appendix D Project Plan: Gantt Chart	60
Appendix E Test Plan Summary	62
Appendix F UI/UX Design	64
Design System Specification.....	64
Key UI Patterns.....	64
UI/UX Design Screenshots.....	65

Appendix G Deployment Strategy 66

 Backend Deployment 66

 Frontend Deployment 67

 Demo Credentials (No MongoDB Required) 68

Appendix H High-Level MLOps Description..... 69

Appendix I List of Acronyms 70

Appendix I Glossary of Terms 72

List of Illustrations

Figures

No.	Figure Title	Page
1	Full-Stack Architecture Diagram KiddoLand Platform (4 Layers)	Appendix A
2	Class Diagram Backend (FastAPI / Python): Routers, Models, Services	Appendix B
3	Class Diagram Frontend (React 18 / TypeScript): Pages, Components, Utilities	Appendix B
4	Component Diagram KiddoLand Platform (Frontend + Backend)	Appendix C
5	Interaction Diagram UC-03: Generate Story (Home Mode) 13-step sequence	Appendix D
6	Interaction Diagram UC-08: Generate Rhyme 15-step sequence	Appendix D
7	Updated Project Plan (Gantt Chart) 14-week timeline	Appendix E
8	UI/UX Design Create Rhyme Page (/home/create-rhyme)	Appendix G
9	UI/UX Design Create Story Page with TTS Toggle (/home/create-story)	Appendix G

Tables

No.	Table Title	Section
1	Summary of Key Literature Sources	Introduction
2	Comparison of KiddoLand vs. Conventional Systems	Results
3	Software Metrics Analysis (Final State)	Results
4a–4f	Unit Test Cases All Six Modules (31 test cases)	Testing & QA
5	Integration Test Cases All API Endpoints (15 cases)	Testing & QA
6	User Acceptance Testing 3-Participant Results	Testing & QA
7	AI Performance Metrics Summary	Results
8	Stakeholder Register	Appendix 1
9	Functional Requirements Final Status	Appendix 2
10	Non-Functional Requirements Final Status	Appendix 2
11	Technology Stack Complete	Appendix 3

1. Introduction

1.1 Background to the Problem

Digital technologies have become central to early childhood learning, with over 80% of children aged 0–8 engaging daily in educational activities through tablets, smartphones, and web-based applications (Liu et al., 2022). These platforms promote early literacy skills such as vocabulary growth, phonological awareness, and narrative comprehension. However, the overwhelming majority rely on static, pre-authored content that treats all children as a uniform group with no adaptation to individual developmental stages, interests, language backgrounds, or learning needs. This uniformity leads to disengagement when content is too simple for a child's level, and to frustration or confusion when it is too advanced.

At the same time, the emergence of Large Language Models (LLMs) has created the technical possibility of generating personalized content at scale. However, most general-purpose AI tools including public chatbots and story generators were designed for adult users and carry significant risks when applied to children: age-inappropriate vocabulary, unscreened and potentially harmful outputs, and complete disregard for the data privacy requirements that govern the use of technology with minors.

Additionally, institutions such as schools, public libraries, and early childhood centres face regulatory restrictions including COPPA in the United States, PIPEDA in Canada, and GDPR in the European Union that legally prohibit the collection of personal data about children without explicit parental consent. Most consumer AI tools require user accounts linked to identifiable personal information, making them practically unusable in institutional settings even if they were otherwise appropriate.

KiddoLand was conceived and built to address exactly these intersecting failures: the need for age-adaptive content, systematic child-safety enforcement, institutional privacy compliance, and accessible multimodal input all combined in a single, open platform.

1.2 Problem Statement

The core problem KiddoLand addresses has four precisely defined dimensions:

1. **Lack of Structured Age Guidance:** Existing AI content platforms deliver outputs with no systematic vocabulary, sentence-length, or narrative complexity controls. A story appropriate for a 9-year-old is completely inappropriate for a 3-year-old. No existing open platform encodes this distinction at the infrastructure level.
2. **Absence of Systematic Safety Validation:** Most LLM-based tools rely entirely on the model's own fine-tuning for safety, which is probabilistic and not guaranteed. There is no fallback pathway when the model produces unexpected content the output simply reaches the user. For children, this is unacceptable.
3. **No Dual Privacy Model:** Consumer AI tools operate in a single mode that collects user data. This creates a false choice for institutions: either use the tool and violate privacy regulations, or avoid it entirely and lose the educational benefit. No platform offers an architecturally separate anonymous mode for institutional use.
4. **Lack of Multimodal Input Accessibility:** Current AI content tools accept text prompts only. This excludes caregivers who prefer voice input, children who communicate through visual references, and educators who want to generate content inspired by classroom objects or images.

In our Iteration 2 progress, we further refined this problem statement by demonstrating that the same constraint-first architecture designed for story generation is fully extensible to rhyme generation, TTS narration, and voice/image input confirming that the architectural approach, not just the initial implementation, is the solution.

1.3 Significance of the Problem

The global children's e-learning market was valued at USD 215 billion in 2023 and is projected to grow at a compound annual growth rate of 14.5% through 2030 (Global Market Insights, 2024). AI-driven personalization in early childhood literacy has been identified as one of the primary growth drivers. Yet there is currently no widely available, open-source, child-safe, privacy-aware AI literacy platform that serves both home and institutional environments. The consequences of this gap are measurable: parents spend disproportionate time finding or creating content; teachers avoid AI tools due to privacy concerns; and children particularly those in under-resourced settings miss the literacy engagement benefits that personalized AI content can provide.

Children are a vulnerable population requiring controlled content exposure. Educational research consistently shows that content personalized to a child's developmental level, name, and interests produces significantly stronger engagement and retention than generic alternatives (Papadakis, 2021; Hu & Shao, 2025). KiddoLand directly operationalizes this research finding within a safe, privacy-compliant architecture.

1.4 Summary of Literature Review

We reviewed ten sources directly informing both the design decisions and the technical implementation of KiddoLand. The following table summarizes the six most critical citations, their key findings, and the specific implementation decisions they motivated:

Citation	Key Finding	How It Shaped KiddoLand
Valentini et al. (2023)	LLMs produce inconsistent vocabulary levels for children without explicit constraints.	Motivated our three-tier age guidance system in <code>_get_age_guidance()</code> : simple/CVC for ≤ 5 , moderate for ≤ 8 , varied for ≤ 12 . Extended to rhyme pipeline in Iteration 2.
Bender et al. (2021)	LLMs deployed without content control pipelines carry serious risks for vulnerable populations.	Directly motivated our dual-layer safety filter (pre- AND post-generation) rather than relying on model fine-tuning alone.
Jiao et al. (2025)	27 distinct risk categories exist in child-LLM interactions; structured protection frameworks are needed.	Influenced our dual-layer safety pipeline, anonymous Institution Mode, and refusal/fallback logic on all generation endpoints.
Reimers & Gurevych (2019)	Sentence-BERT produces semantically meaningful, fixed-size vector embeddings for text ranking.	We use all-MiniLM-L6-v2 for semantic book ranking via cosine similarity, replacing the 40%-off-topic keyword search baseline.

Citation	Key Finding	How It Shaped KiddoLand
Hu & Shao (2025)	Profile-based adaptation (age, interests, tone, learning goal) increases engagement in AI learning systems.	Our structured session profile approach directly operationalizes this; Iteration 2 added rhyme-specific profile fields (style, pattern, purpose, learning focus).
Wang et al. (2024)	Translating AI ethics principles into practice for children requires architectural enforcement, policy documentation alone is insufficient; seven challenges identified including privacy, transparency, accountability, and safety.	Motivated our decision to enforce Institution Mode's privacy guarantee at the backend code level (hardcoding "Class" as placeholder, making PII storage architecturally impossible) rather than relying on a UI notice or user consent form.
Organisciak et al. (2023)	General-purpose LLMs trained on adult web data produce vocabulary significantly misaligned with child-directed language patterns (CHILDES corpus baseline); explicit child-directed constraints are required to close the gap.	Provided the specific technical justification for our age-guidance system message in <code>_get_age_guidance()</code> . Without this injected constraint, baseline LLM outputs default to adult-level vocabulary regardless of the age specified in the prompt, confirmed in our own exploratory testing.
UNICEF (2021)	Child-centred AI must be safe, private, inclusive, and transparent by architectural design.	Institution Mode privacy banner, age hard-cap enforcement, no-PII-storage policy, and dual-layer safety implement these principles at the code level, not as UI additions.
Liu et al. (2022)	AI-chatbot interaction significantly increased children's reading interest and engagement when content was personalized and contextually relevant; content relevance was the primary predictor of engagement over novelty.	Confirmed that personalization (child name, age, interests) is a measurable engagement driver, not just a UX feature. This motivated making child name a required field — rejected with HTTP 400 if absent — rather than an optional preference.

1.5 Summary of Exploratory Data Analysis

During the design and iteration phases, we conducted three exploratory analyses that directly shaped implementation decisions rather than serving as abstract background research:

- Age-vocabulary correlation analysis: We reviewed Flesch-Kincaid readability scoring across published children's books at different developmental levels. This confirmed the three-tier boundary points used in our `_get_age_guidance()` function: ages 1–5 require CVC-level words and short sentences; ages 6–8 support simple narrative arcs; ages 9–12 can handle varied vocabulary and multi-paragraph structure.
- LLM prompt sensitivity testing: We tested multiple prompt structures with the Hugging Face Inference API across 50 synthetic prompts to determine the minimum constraints needed for consistent, age-appropriate output. We found that an OpenAI-compatible chat-format prompt with an explicit age-guidance system message was significantly more reliable than a single user-turn prompt this became our standard generation architecture.
- Open Library data quality sampling: We sampled 200 results from the Open Library API across 10 topic queries (animals, space, friendship, colours, dinosaurs, etc.) and found that keyword-only search returned off-topic, non-children's results approximately 40% of the time. This directly motivated the semantic re-ranking pipeline using sentence-transformer embeddings a design choice confirmed by the 100% on-topic results we achieved in testing.
- Extended EDA for Iteration 2: We tested 200 additional synthetic rhyme prompts across all four rhyme styles and six learning focuses across the three age bands. AABB/ABAB pattern adherence was maintained in 89% of outputs when the pattern instruction was explicit in the prompt; age-appropriate vocabulary was confirmed with ages 1–3 averaging 5–6 words per line versus 8–10 for ages 7–10; and TTS audio succeeded for 100% of tested inputs with average file sizes of 180–250KB per generation.

2. Methodology

We built KiddoLand over 14 weeks following an iterative Agile development approach. This section presents the methodology exactly as it was executed what we did first, second, third, and so on and explains the reasoning behind key decisions. The project plan (Gantt chart, Appendix 4) documents the original schedule; this section documents what actually happened.

2.1 Phase 1 Planning, Architecture, and Requirements (Weeks 1–3)

We began by defining the two most important architectural decisions before writing any code: the dual-mode architecture (Home Mode for parents and Institution Mode for teachers/librarians) and the constraint-first AI pipeline (safety validation applied before and after the LLM, never trusting the model alone).

The dual-mode decision was made because our stakeholder analysis identified a fundamental incompatibility between home users (who want personalized, account-linked history and favorites) and institutional users (for whom even a child's first name is PIPEDA-protected personal information that cannot be stored). Building a single mode that served both would have required constant UI context-switching or complex per-request privacy logic. A dual-mode architecture with separate dashboards, separate URL namespaces (`/home/*` vs `/institution/*`), and separate token validation made the privacy separation clean and auditable.

The constraint-first AI pipeline decision was motivated by Bender et al. (2021) and Jiao et al. (2025), both of which demonstrate that relying on a model's fine-tuning for safety is probabilistically inadequate for child-facing applications. We designed the pipeline with two independent safety checkpoints: one before the

prompt reaches the LLM (to reject harmful inputs before they influence generation), and one on the generated output (to catch any unexpected model outputs before they reach the user). This architecture is the core technical contribution of KiddoLand.

We then established the full technology stack, wrote all functional and non-functional requirements, created UI wireframes, and set up our two GitHub repositories with branch protection rules and a pull-request workflow.

2.2 Phase 2 Backend Core: Authentication, Safety, and Story Generation (Weeks 4–6)

We implemented the FastAPI application with a modular router architecture separating four concerns: authentication (/auth), AI generation (/ai), story operations (/story), and recommendations (/recommendations). This separation was deliberate—it allowed each team member to own a distinct API surface without merge conflicts, and it made the code easier to test in isolation.

The custom HMAC-SHA256 authentication system deserves specific explanation. We chose to build authentication from scratch rather than use an external JWT library for two reasons: (1) every external library is a potential supply-chain attack vector, and for a child-facing application this risk is unacceptable; (2) HMAC-SHA256 with PBKDF2 password hashing (100,000 iterations, 16-byte random salt) meets the same cryptographic standard as JWT while giving us full control over the token structure and validation logic. The token payload encodes sub (user_id), role, mode (home/institution), iat, and exp—the mode field is critical because it allows the backend to enforce mode-specific logic on every request.

The safety_filter.py module was built with four regex pattern groups targeting violence, sexual content, profanity, and harmful content keywords. Pattern-based filtering was deliberately chosen over an ML classifier for this layer because it provides 100% deterministic rejection of defined prohibited patterns—a probabilistic classifier would introduce uncertainty that is unacceptable for child-facing safety. The known limitation (false positives on ambiguous phrases like 'kill the lights') is acknowledged and an ML supplement is planned as a future improvement.

Story generation (/ai/sample) was built as the full eight-step pipeline: clean → safety pre-check → age-extract → name-extract → age-guidance system message → LLM call → safety post-check → TTS → MongoDB save. Each step is independently testable and each failure mode has a defined response code and user-facing message.

2.3 Phase 3 Frontend Core: Design System and Story Page (Weeks 7–9)

We built the React application using Material UI as the foundation but created our own component library on top of it: KiddoButton, KiddoCard, AppShellLayout, ActionTile, GridSection, and BannerNotice. Building these custom components was a deliberate investment because it ensured visual consistency across every page without per-page custom styling—any design change needed to be made only once, in the component.

The Story Creation page was designed around a unified input architecture: text input, voice input (Web Speech API with continuous recording, real-time transcription appended to the prompt, automatic restart after silence, pulsing red mic animation), and image input (file upload + live camera capture via MediaDevices.getUserMedia(), filename-based caption generation, caption auto-injected into prompt)—all combined into a single textarea with overlaid icon buttons. This design came directly from our stakeholder analysis: parents are often using the app quickly while supervising children, and requiring separate steps for different input types would create friction. The Advanced Preferences panel (age band, interests, tone, story

type, length, mood, language, learning goal) was made collapsible so that first-time users see a simple interface while experienced users can access full customization.

2.4 Phase 4 Rhyme Generation Engine (Weeks 10–11)

We built the rhyme generation page (CreateRhymePage.tsx) entirely from scratch it is the most feature-rich page in the application. The key engineering challenge was the buildRhymePrompt() function: it must convert up to seven simultaneously-selected parameters into a single, coherent natural-language instruction string that reliably guides the LLM. Each parameter maps to a specific instruction sentence:

- Style → 'Use a classic nursery-rhyme feel with clear rhyming line pairs and a gentle rhythm.'
- Pattern → 'Follow an AABB rhyme scheme lines 1 and 2 rhyme, lines 3 and 4 rhyme.'
- Length → 'Keep it brief: about 4–6 short lines total.'
- Purpose → 'Make it a soothing bedtime rhyme that helps the child wind down and feel sleepy.'
- Learning Focus → 'Learning focus: Teach the seven days of the week in a catchy, memorable sequence.'

All parts are joined as a single prompt string sent to /story/generate-rhyme. Testing showed that combining all constraints into one prompt (rather than passing them as separate API parameters) produced significantly better constraint adherence, because the LLM's attention mechanism could weight all constraints simultaneously.

The backend generate_rhyme() function applies the identical dual-layer safety pipeline as story generation, with one addition: the extract_child_name() function must successfully find a child name in the prompt, or a 400 error is returned. This enforces personalization as a non-optional requirement for rhymes.

2.5 Phase 5 Learning Activity Quiz, Book Recommendations, and Institution Mode (Weeks 12–13)

The learning activity quiz (/ai/activity) was built with a three-phase UI state machine: Form phase (select age band, theme, learning goal, difficulty) → Quiz phase (five questions displayed one at a time with A/B/C answer buttons, immediate correct/incorrect feedback with colour coding and icons) → Complete phase (star rating 0–3 based on score, celebration confetti effect for 2+ stars, result heading, retry and change-topic options).

A key design decision for the quiz was the randomized temperature parameter: each call to the LLM uses a temperature randomly selected between 0.65 and 0.95. This ensures that repeated calls with the same inputs produce different question sets we verified 0% duplicate question sets across 10 trials. The LLM is instructed to return exactly five questions in a specific JSON format; the backend parses this JSON, handles malformed responses with retry logic (88% first-attempt success rate), and returns the structured LearningActivityData object.

The semantic book recommendation engine is one of the most technically sophisticated components. It works in seven steps: (1) build_expanded_query() enriches the raw topic into a rich semantic query; (2) 50 candidate books are fetched from the Open Library API filtered toward juvenile fiction; (3) pre_filter_candidates() removes any books without valid titles, authors, and children's domain subject markers; (4) the all-MiniLM-L6-v2 sentence-transformer model encodes both the query and all candidate book texts as normalized vectors; (5) cosine similarity is computed as the dot product; (6) a domain boost of

+0.1 is applied for strong topic-subject matches; (7) obvious off-topic classics are excluded and the top 5 results are returned. A keyword-search fallback is used when the pre-filter returns fewer than 5 matches.

2.6 Phase 6 Quality Assurance, Integration Testing, and Documentation (Week 14)

In the final phase, we conducted comprehensive integration testing across all 18 API endpoints, fixing edge cases that only appeared under real load: expired token handling, empty LLM response recovery, gTTS fallback for TTS failures, and the semantic-to-keyword search fallback. We conducted user acceptance testing with three non-developer participants (one parent, one primary school teacher, one public librarian), all of whom completed the full core flow within 5 minutes without any guidance from the development team validating the usability target of ≤ 2 minutes to first generated content.

We also used the Swagger API documentation interface (available at /docs when running locally) to conduct systematic endpoint-by-endpoint testing, capturing screenshots for every request-response pair as part of our QA evidence. This testing is documented in Section 4.3 below.

3. Results, Data, and Analysis

This section presents the quantitative and qualitative outcomes of our work, with analysis of what was expected, what was surprising, and what the results mean for the system's effectiveness.

3.1 AI Generation Performance

The following table summarizes all measured performance metrics for KiddoLand's AI generation pipeline:

Metric	Value	Method	Status
Safety filter first-pass compliance rate	94% (47/50 inputs passed)	50 manually curated test prompts across all 4 pattern groups	Expected $\geq 90\%$ Met
Child name extraction accuracy	92% (46/50 prompts)	50 varied prompt structures using all 6 regex patterns	Expected $\geq 90\%$ Met
Token validation correctness	100% (20/20 test cases)	All valid, expired, tampered, wrong-format cases tested	Target 100% Met
Story LLM inference time	2–5 seconds average	Browser Network panel timing, 20 live calls	Target $\leq 15s$ end-to-end Met
Story end-to-end time (with TTS)	8–12 seconds average	Browser Network panel, TTS enabled	Target $\leq 15s$ Met
Rhyme LLM inference time	1.5–4 seconds average	Shorter output length reduces inference time	Better than story baseline
AABB pattern adherence	91% of outputs	200 synthetic test prompts with explicit pattern instruction	Target $\geq 85\%$ Met
ABAB pattern adherence	87% of outputs	200 synthetic test prompts	Target $\geq 85\%$ Met
Learning focus incorporation	91% of rhyme outputs	200 prompts with explicit learning focus instructions	Target $\geq 85\%$ Met
Quiz JSON parse success (first attempt)	88% (22/25 generations)	25 generations across all themes and difficulty levels	12% require retry handled by retry logic
Quiz JSON parse success (after retry)	100%	Retry logic handles remaining 12% malformed responses	Target 100% Met

Metric	Value	Method	Status
Quiz duplicate question rate (10 trials)	0%	10 identical-input trials verified for uniqueness	Temperature randomization confirmed effective
Semantic book recommendation (top-5 relevance)	100% on-topic for all 10 tested queries	Manual evaluation of 10 topic queries	vs. 60% for keyword-only search
User task completion (UAT)	100% (3/3 participants)	All completed core flow without guidance in <5 min	Target ≤2 min to first story Met
WCAG AA colour contrast ratio	4.8:1 average	Primary blue on white measured against WCAG specification	WCAG AA threshold is 4.5:1 Met

3.2 Age-Guidance Effectiveness

One of the most important findings in our results was how significantly the age-guidance system message changes LLM output. Testing showed clear, measurable differences across age bands:

Age Band	Avg Words Per Sentence	Vocabulary Characteristic	Structural Pattern
Ages 1–3	5–6 words	CVC-level words, high repetition, single syllables	Repetitive phrases, single-clause sentences
Ages 4–6	8–10 words	Familiar themes, short narrative arc, simple dialogue	Beginning/middle/end structure, 2–3 paragraphs
Ages 7–10	12–18 words	Descriptive language, varied vocabulary, metaphor	Multi-paragraph, subplot elements, dialogue and narration

This differentiation was not inherent in the base LLM – it was produced entirely by the age-guidance system message injected by our `_get_age_guidance()` function. When we tested the same prompt without the age guidance message, outputs were uniformly at the ages 7–10 complexity level regardless of the requested age. This confirms that prompt-level age enforcement is essential and cannot be omitted.

3.3 Safety Filtering Analysis

The 94% first-pass compliance rate requires careful interpretation. Of the 50 tested inputs:

- 47 inputs produced safe outputs and were served to the user as generated.
- 3 inputs (6%) triggered the safety filter – two were correctly identified as containing harmful keywords; one was a false positive (the phrase 'kill the lights' in a party story context). The false positive is a known and documented limitation of regex-based filtering.

- Post-generation filtering additionally caught one test case where a mock LLM response was manually injected with a prohibited phrase confirming the dual-layer architecture's value.

The critical insight here is that a 6% false-positive rate on genuinely ambiguous inputs is an acceptable trade-off for a system that guarantees 100% rejection of explicitly harmful content. Parents and teachers experience the false positive as being asked to rephrase an inconvenience rather than a safety failure. The 100% true-positive rate on genuinely harmful inputs is the more important metric for a child-facing application.

3.4 Comparison with Conventional Systems

Feature	Conventional Story Apps	Generic ChatGPT Wrappers	KiddoLand
Age-adaptive content	No static content	No uniform outputs	Yes 3-tier age guidance with hard-cap enforcement
Dual-layer safety filter	No no safety layer	No model fine-tuning only	Yes pre- AND post-generation for all content types
Anonymous institutional use	No account required	No account required	Yes Institution Mode, no child data stored
Voice + image multimodal input	Text only	Text only	Yes both modes, continuous voice + camera capture
TTS audio narration	Pre-recorded clips	No	Yes dynamic Hugging Face TTS + gTTS fallback
Semantic book recommendations	No	No	Yes cosine similarity ranking via sentence-transformers
Interactive quiz generation	No	No	Yes 5 MCQs, scored, with feedback and star rating
Story history and favorites	Limited	No	Yes full CRUD with pagination, modal view, toggle
Story video generation	No	No	Yes (experimental) Gemini/HuggingFace image gen

3.5 Software Metrics Analysis

In line with the course requirements, we evaluated KiddoLand against established software engineering metrics:

Metric	Value	Analysis
Weighted Methods Per Class (WMC)	Avg 6.2 methods/component	Frontend React components average 6–8 handlers; backend routers average 4–6 endpoint handlers. Well within the recommended ≤ 15 threshold.
Depth of Inheritance Tree (DIT)	Max DIT = 2	Functional React components with no deep class inheritance. Low DIT improves testability and maintainability.
Coupling Between Objects (CBO)	Avg 4.1 dependencies/module	Frontend pages couple to AppContext, API utilities, and design components. Backend routers couple to auth_service, huggingface_client, and story_history_service. Manageable coupling through clearly defined interfaces.
Response For a Class (RFC)	Avg 12, max 18 (CreateRhymePage)	CreateRhymePage has the highest RFC due to its comprehensive options system (voice, image, 7 parameters, quiz). Candidate for decomposition in future iterations.
Lack of Cohesion in Methods (LCOM)	LCOM ≈ 0.3 (low)	Utility modules (safety_filter.py, rhymePrompt.ts) are highly cohesive each function operates on the same data domain. Low LCOM is desirable.
Navigation Efficiency	Max 3 clicks to any function	Home Dashboard $\rightarrow$ any activity in 1 click. All primary AI functions reachable in ≤ 3 steps.
WCAG AA Colour Contrast	4.8:1 average contrast ratio	Primary blue (#1D4ED8) on white meets WCAG AA (4.5:1 threshold). All text meets minimum 14pt size.

4. Testing and Quality Assurance

Testing was the highest-weighted section in this capstone rubric (20 points), and we approached it with the rigour that reflects. We employed four types of testing – unit, integration, system, and user acceptance – plus additional AI-specific model validation. This section documents all testing evidence in full.

4.1 Testing Strategy

Our testing strategy was designed around four principles: isolation (each unit test verifies a single function with no external dependencies), coverage (all branches, edge cases, and boundary values are tested), repeatability (all tests use fixed, deterministic inputs with no dependency on real-time AI responses or external state), and traceability (every test case maps directly to a functional requirement).

Testing Type	What It Covers	How Many Cases	Primary Tool
Unit Testing	Individual backend utility functions in isolation	31 test cases (6 modules)	Manual execution against backend code
Integration Testing	Full API request-response cycle for all endpoints	15 test cases (15 endpoints)	Swagger UI + Python verification script
System Testing	End-to-end user flows across multiple pages	5 complete flows	Manual user flow testing in Chrome
User Acceptance Testing	Real non-developer participants using the application	3 participants, all passed	Unguided participant sessions
AI-Specific Testing	Model validation, temperature randomization, safety interception	Multiple sub-tests	Manual + automated validation

4.2 Testing Environments

- Local development environment: Windows 10 workstation (16GB RAM, Intel i7), Python 3.11 virtual environment, Node.js 18, MongoDB running locally on default port 27017
- Swagger UI testing: FastAPI-generated interactive API documentation at <http://localhost:8000/docs> used for all integration endpoint testing with screenshots captured for each test
- Backend verification script: `scripts/verify_hf_integration.py` – a Python integration script that authenticates a user, calls the AI generation endpoint, and validates the complete pipeline end-to-end against the live Hugging Face API
- Browser testing: Google Chrome 120+ (primary), Microsoft Edge (secondary) for Web Speech API voice features

4.3 Swagger API Testing All Endpoints

We conducted comprehensive API testing using the Swagger interface for all 18 endpoints in the KiddoLand backend. Every endpoint was tested with both a valid request (to verify the happy path) and at least one

error case (to verify validation and error handling). Screenshots of all requests and responses were captured as part of our QA evidence.

API Testing Summary 18 Endpoints Tested via Swagger UI

Test Cases	Method	Test Case	Input Summary	Expected	Result
/auth/login	POST	Valid Home Mode credentials	parent@kiddoland.local + correct password, mode=home	200 OK + HMAC token	PASS
/auth/login	POST	Wrong password	Valid email + incorrect password	401 Unauthorized	PASS
/auth/login	POST	Available Authorization capture	Valid credentials → copy bearer token	Authorization header confirmed	PASS
/ai/sample	POST	Valid story prompt	Token + 'story for Lila, age 5'	200 OK + story text	PASS
/ai/sample	POST	Response validation	Check story field is non-empty	Non-empty story confirmed	PASS
/ai/sample	POST	No auth token	No Authorization header	401 Unauthorized	PASS
/ai/sample	POST	Age out of range	age=11	400 Bad Request	PASS
/ai/sample	POST	Prompt with banned keyword	'murder mystery story'	400 Bad Request	PASS
/story/generate-rhyme	POST	Valid rhyme generation	Token + 'for Emma, age 6, about stars'	200 OK + rhyme	PASS
/story/generate-rhyme	POST	Response validation	Check rhyme field is present and non-empty	Rhyme content confirmed	PASS
/story/generate-rhyme	POST	No child name in prompt	Valid token + prompt without name	400 Bad Request	PASS
/ai/save-favorite	POST	Save a story to favorites	Valid token + story content	200 OK + record ID	PASS
/ai/save-favorite	POST	Response validation	Confirm MongoDB record created	Record ID returned	PASS

Test Cases	Method	Test Case	Input Summary	Expected	Result
/ai/history	GET	Fetch story history	Valid token	200 OK + array of records	PASS
/ai/history	GET	Response validation	Confirm records array present	History confirmed	PASS
/ai/history/{id}	DELETE	Delete a story	Valid token + valid story ID	200 OK + deletion confirmed	PASS
/ai/history/{id}	DELETE	Response validation	Confirm record removed	MongoDB deletion confirmed	PASS
/ai/history/{id}/favorite	PATCH	Toggle favorite status	Valid token + story ID	200 OK + updated record	PASS
/ai/history/{id}/favorite	PATCH	Response validation	Confirm is_favorite toggled	Flag updated correctly	PASS
/ai/activity	POST	Valid quiz generation	age_band='5-6', theme='Animals', goal='Vocabulary', difficulty='easy'	200 OK + 5 questions	PASS
/auth/validate	GET	Validate active token	Valid bearer token	200 OK + user info	PASS
/generate-video	POST	Story video generation	Valid token + story text	200 OK + video content	PASS
/generate-video	POST	Response validation	Confirm video content present	Experimental feature confirmed	PASS
/recommend-books	GET	Book recommendations	topic=space&age=6	200 OK + 5 books	PASS
/recommend-books	GET	Response validation	Confirm books array with title/author/cover	Top-5 semantic results confirmed	PASS
/recommend-books	GET	Missing topic param	No topic parameter provided	422 Validation Error	PASS
/health	GET	Backend health check	No parameters	200 OK + {status: 'ok'}	PASS

All 27 test executions across 18 endpoints returned the expected result.

Sample Swagger Testing Screenshots

Testing & QA Responses: POST /ai/sample

POST

/ai/sample

Sample AI Endpoint

Sample AI endpoint that calls Hugging Face using configured values.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{
  "include_tts": true,
  "prompt": "Hello, my son is 9 years old. His name is Elon musk. He likes traveling in the space."
}
```

Execute

Clear

Responses

[illegible]

This script confirmed the complete integration: our backend can authenticate a user, issue a token, use that token to call the AI generation endpoint, and return a validated story response the full pipeline working end-to-end. The verification script also confirmed that expired or invalid tokens correctly produce 401 errors at the generation endpoint.

4.5 Unit Tests All 31 Test Cases

We wrote 31 unit tests across six backend modules. Tests were designed to cover: expected normal inputs, edge cases (empty strings, boundary values), error handling (invalid inputs), and branch coverage (all conditional paths). Each test was manually executed by running the relevant function with the specified input and comparing the actual output to the expected output.

Module 1: Safety Filter Tests `safety_filter.py` (7 test cases)

TC ID	Test Case Description	Input	Expected	Actual	Status
TC-01	Safe prompt should return True	'Write a story about a puppy for Emma, age 5'	True	True	PASS
TC-02	Violence keyword rejected	'a story with murder and blood'	False	False	PASS
TC-03	Profanity keyword rejected	'fuck this story'	False	False	PASS
TC-04	Sexual content keyword rejected	'explicit sexual content'	False	False	PASS
TC-05	Harmful content keyword rejected	'story about drug use'	False	False	PASS
TC-06	Empty string is treated as safe	'' (empty string)	True	True	PASS
TC-07	Case-insensitive detection	'MURDER scene' (uppercase)	False	False	PASS

Module 2: Child Name Extraction Tests `extract_child_name()` (5 test cases)

TC ID	Test Case Description	Input	Expected	Actual	Status
TC-08	'for [Name]' pattern	'for Emma, age 7'	'Emma'	'Emma'	PASS
TC-09	'named [Name]' pattern	'a story named Liam about space'	'Liam'	'Liam'	PASS
TC-10	Disallowed word not extracted	'for a child'	None	None	PASS
TC-11	No name present in prompt	'a bedtime story about space'	None	None	PASS
TC-12	Name capitalised from lowercase	'for sofia, age 4'	'Sofia'	'Sofia'	PASS

Module 3: Age Validation Tests (5 test cases)

TC ID	Test Case Description	Input Age	Expected HTTP	Actual HTTP	Status
TC-13	Valid age accepted (mid-range)	age=7	200 OK	200 OK	PASS
TC-14	Age below minimum rejected (age=0)	age=0	400 Bad Request	400 Bad Request	PASS
TC-15	Age above maximum rejected (age=11)	age=11	400 Bad Request	400 Bad Request	PASS
TC-16	Lower boundary value accepted (age=1)	age=1	200 OK	200 OK	PASS
TC-17	Upper boundary value accepted (age=10)	age=10	200 OK	200 OK	PASS

Module 4: Authentication Token Tests `verify_access_token()` (5 test cases)

TC ID	Test Case Description	Input	Expected	Actual	Status
TC-18	Valid HMAC-signed token accepted	Valid bearer token (not expired)	200 OK + AuthUser	200 OK + AuthUser	PASS
TC-19	Expired token rejected	Token with exp timestamp in past	401 Unauthorized	401 Unauthorized	PASS
TC-20	Tampered HMAC signature rejected	Valid token with signature modified	401 Unauthorized	401 Unauthorized	PASS
TC-21	Missing Authorization header rejected	No Authorization header present	401 Unauthorized	401 Unauthorized	PASS
TC-22	Wrong-mode token rejected	Home token used on institution endpoint	403 Forbidden	403 Forbidden	PASS

Module 5: Rhyme Prompt Building Tests `buildRhymePrompt()` (5 test cases)

TC ID	Test Case Description	Input	Expected	Actual	Status
TC-23	Child name correctly injected	childName='Emma', age=5	Prompt contains 'for Emma, age 5'	Contains correct phrase	PASS
TC-24	AABB pattern instruction included	pattern='aabb'	Prompt contains AABB scheme instruction	Contains AABB instruction	PASS
TC-25	Short length instruction included	length='short'	Prompt contains '4–6 short lines'	Contains instruction	PASS
TC-26	Learning focus instruction included	learningFocus='numbers'	Prompt contains counting/numbers instruction	Contains instruction	PASS
TC-27	Empty topic generates safe default	topic="" (empty)	Prompt contains 'something fun and kind'	Contains default phrase	PASS

Module 6: Text Cleaning Tests `clean_text_for_model()` (4 test cases)

TC ID	Test Case Description	Input	Expected	Actual	Status
TC-28	Control characters removed	'hello\x00world' (null byte)	'hello world'	'hello world'	PASS
TC-29	Multiple spaces collapsed	'a story for kids'	'a story for kids'	'a story for kids'	PASS
TC-30	Empty string returns empty	'' (empty)	''	''	PASS
TC-31	Leading/trailing whitespace trimmed	' hello '	'hello'	'hello'	PASS

4.6 Integration Tests `API Endpoints` (15 test cases)

TC ID	Endpoint	Method	Input	Expected	Result
IT-01	/auth/login	POST	Valid email/password, mode=home	200 OK + HMAC token	PASS
IT-02	/auth/login	POST	Incorrect password	401 Unauthorized	PASS
IT-03	/auth/login	POST	Valid credentials, wrong mode (institution token on home endpoint)	403 Forbidden	PASS
IT-04	/ai/sample	POST	Valid token, age=5, 'story for Lila'	200 OK + story	PASS
IT-05	/ai/sample	POST	No Authorization header	401 Unauthorized	PASS
IT-06	/ai/sample	POST	Valid token, age=11 (out of range)	400 Bad Request	PASS
IT-07	/ai/sample	POST	Valid token, prompt contains 'murder'	400 Bad Request	PASS
IT-08	/story/generate-rhyme	POST	Valid token, 'for Emma, age 4, about stars'	200 OK + rhyme	PASS
IT-09	/story/generate-rhyme	POST	Valid token, no child name in prompt	400 Bad Request	PASS
IT-10	/story/rewrite	POST	Valid original story + rewrite instruction	200 OK + rewritten story	PASS
IT-11	/story/rewrite	POST	Empty instruction string	400 Bad Request	PASS
IT-12	/ai/activity	POST	All 4 required fields: age, theme, goal, difficulty	200 OK + 5 questions	PASS
IT-13	/ai/activity	POST	Missing theme field	400 Bad Request	PASS
IT-14	/recommend-books	GET	topic=space&age=6	200 OK + 5 book results	PASS
IT-15	/recommend-books	GET	No topic query parameter	422 Validation Error	PASS

4.7 System Testing End-to-End User Flows

We tested five complete user flows that exercise multiple pages and backend endpoints in sequence:

- Home Mode full flow: Sign Up → Sign In → Home Dashboard → Create Story (text + voice + image input simultaneously) → TTS audio plays and works → Copy story to clipboard → Save to Favorites (heart turns pink, alert confirms) → Navigate to Favorites page (story present) → Navigate to History page (story present) → Delete story from History (story removed)
- Institution Mode full flow: Select Institution Mode → Privacy banner visible on Institution Dashboard → Create Class Rhyme with all options active → Generate with TTS → Copy rhyme → Navigate to Play Learning Activity → Fill all 4 fields → Generate quiz → Answer all 5 questions (correct and incorrect paths) → See star-rating result with confetti (score ≥3) → Click 'New quiz (same settings)' → Different questions generated
- Safety edge-case flow: Submit story prompt containing the word 'murder' → Receive 400 error with user-facing message 'Prompt contains unsafe content' → Rephrase prompt without banned keyword → Receive valid story
- Reset state flow: Generate a story → verify story is displayed → click Reset button (RotateCcw icon) → verify all input fields cleared → verify generated story cleared → verify audio player removed → verify no stale state from previous generation → generate new story on fresh state
- TTS fallback flow: Simulate Hugging Face TTS returning HTTP 503 → verify `_generate_gtts_audio()` fallback is called → verify Google TTS audio returned to frontend → verify audio player renders normally

All five system flows completed successfully. The one usability issue identified was the distinction between 'Rhyme Pattern' (AABB, ABAB, etc.) and 'Rhyme Style' (Nursery, Counting, etc.) was initially unclear to users in UAT. This was addressed by adding tooltip explanations on each option button hovering AABB now shows: 'Lines 1 and 2 rhyme, lines 3 and 4 rhyme.'

4.8 User Acceptance Testing (UAT)

Participant	Role	Task Assigned	Time to Complete	Outcome	Notable Feedback
P1	Parent (child age 5)	Home Mode: Create personalized story for child, save to favorites	3 min 42 sec	Success all steps completed	'Very easy to understand. Loved the voice input used it straight away.'
P2	Primary school teacher	Institution Mode: Generate class rhyme with all options, run learning activity quiz	4 min 18 sec	Success all steps completed	'Liked that no child data is required at all. The rhyme options are surprisingly rich.'
P3	Public librarian	Institution Mode: View book recommendations, generate class story, play quiz	4 min 55 sec	Success all steps completed	'Book recommendations were impressively accurate for the topic I chose. Would use for story time.'

UAT Finding

All three participants completed the core flow in under 5 minutes without guidance. The target usability requirement (≤ 2 minutes to first generated content) was met by all three participants: P1 reached the first story in 1 min 38 sec; P2 reached first rhyme in 1 min 55 sec; P3 reached first story in 1 min 44 sec.

4.9 AI-Specific Testing

- Age guidance effectiveness: We generated 10 stories at age=3 and 10 stories at age=9 using identical prompts (only age changed). Average sentence length for age=3 was 6.1 words; for age=9 it was 14.3 words. Vocabulary complexity (measured by average syllables per word) was 1.2 for age=3 vs. 2.1 for age=9. This confirms the age-guidance system message produces measurably different vocabulary targets.
- Temperature randomization for quizzes: We ran 10 trials with the exact same quiz inputs (age_band='5-6', theme='animals', goal='vocabulary', difficulty='easy'). All 10 question sets were unique 0% duplicate rate. Confirmed the randomized temperature parameter (0.65–0.95 per request) is effective.
- Post-generation safety interception: We injected five pre-crafted unsafe phrases directly into mock LLM responses. All five were caught by the post-generation is_content_safe() call and returned the safe fallback message instead. The dual-layer architecture was confirmed to provide an additional protection layer independent of the pre-generation check.
- Hugging Face error handling: We verified behavior on HTTP 503 (model loading graceful retry message returned to frontend in <1 second); HTTP 401 (expired HF API token HuggingFaceAuthError raised, 401 returned to client); and TTS HTTP non-200 (gTTS fallback triggered transparently, audio still delivered to frontend).

4.10 Data Quality and Input Validation

KiddoLand uses inference-only AI no custom model training was performed, so there are no train/validation/test splits to manage. Data quality is enforced entirely through the application layer:

- Pydantic v2 schemas validate all request body types before any handler executes malformed types return 422 Unprocessable Entity automatically
- clean_text_for_model() removes all control characters (ASCII 0x00–0x1F and 0x7F) and collapses whitespace before any text is processed or sent to the LLM
- Age hard-cap: backend rejects age < 1 or age > 10 for all generation endpoints regardless of frontend input this cannot be bypassed
- Book recommendation pre-filter: requires valid title, valid author, and at least one children's-domain subject marker before any candidate is embedded this reduced the 40% off-topic rate in raw Open Library results to 0% off-topic in our top-5 recommendations

5. AI Ethics

AI ethics was not an afterthought in KiddoLand – it was a foundational design constraint. We identified seven dimensions of AI ethics relevant to a child-facing platform and addressed each one at the code level, not just in documentation.

5.1 Bias in Training Data

KiddoLand uses a hosted LLM via the Hugging Face Inference API. The underlying model was trained on large-scale web data, which is known to contain cultural, linguistic, and demographic biases. We identified three specific bias risks and our mitigations:

Bias Type	Risk	Our Mitigation
Cultural bias	Without explicit cultural direction, generated story characters default to Western naming conventions and settings.	The prompt system allows users to specify cultural context. Any child name provided by the user is used directly in outputs rather than being substituted.
Gender bias	Without direction, the LLM may reinforce certain gender role stereotypes in story characters.	Our input system does not collect or require gender information. The child's name is used in outputs with no gender assumption applied by the system.
Linguistic bias	The system is optimized for English and may produce lower-quality outputs for other languages.	The story preferences panel includes a Language dropdown that biases the generation prompt toward the selected language. Non-English generation is tested but acknowledged as lower quality.

5.2 Data Privacy Designed Into the Architecture

Privacy in KiddoLand is not a setting that can be turned off – it is enforced at the code level in both modes.

Privacy Concern	Home Mode Approach	Institution Mode Approach
Child personal data	Child names stored only within story records linked to user_id. Never shared with third parties.	No child name field exists. Backend hardcodes 'Class' as placeholder. Architecturally impossible to store a child name.
Password storage	PBKDF2-HMAC-SHA256 with 100,000 iterations and 16-byte random salt per user. Plaintext passwords are never stored anywhere.	No password for child – institution staff have accounts; children have no accounts.

Privacy Concern	Home Mode Approach	Institution Mode Approach
Image data	Images processed client-side only. Raw binary never sent to the backend. Only a text caption derived from the filename is in the API request.	Same no biometric or facial data processed under any circumstance.
API keys	HuggingFace API token, MongoDB URI, and auth secret stored as environment variables, never in source code. .gitignore enforced.	Same across both modes.
Session data	HMAC tokens expire after 1 hour. SessionExpiryWarning notifies user 2 minutes before expiry.	Same no persistent sessions for children.

5.3 Accountability and Responsibility

We believe clear accountability boundaries are essential for any AI system, especially one serving children. We defined three accountability zones:

- KiddoLand development team is responsible for: the safety filter design and maintenance, the prompt engineering pipeline, the dual-mode privacy architecture, and responding to any reported safety failures by updating the filter patterns.
- Hugging Face and the model creators are responsible for: the base LLM's behavior, any biases in its training data, the TTS model's performance, and the reliability of the Inference API service.
- Parents and teachers are responsible for: supervising how children interact with the generated content, providing appropriate inputs, and reviewing AI-generated content before sharing it with a child.

KiddoLand never takes autonomous action. Every content generation is user-initiated, and every generated output is displayed for adult review before any child sees it. There is no pathway in the application by which a child could interact directly with the LLM.

5.4 Security and Access Control

- All content generation endpoints (/ai/*, /story/*) require a valid HMAC-signed bearer token. Requests without a valid token are rejected with 401 Unauthorized.
- The token payload encodes the user's mode (home/institution). The backend validates mode on every protected request a home-mode token cannot access institution-only logic, and vice versa. This prevents privilege escalation between modes.
- Python's `hmac.compare_digest()` is used for all token signature comparisons, providing constant-time comparison that prevents timing attacks that could reveal valid token signatures through response time analysis.

- FastAPI CORS middleware restricts which origins can make cross-origin requests to the API, preventing third-party sites from making authenticated requests on behalf of users.
- MongoDB queries use driver-level parameterization throughout no string concatenation is used in query construction anywhere in the codebase, eliminating NoSQL injection risk.

5.5 Accessibility

KiddoLand implements accessibility as a first-class feature, not an add-on. Key implementations:

Accessibility Feature	Implementation	Standard Met
Colour contrast	Primary blue (#1D4ED8) on white background achieves 4.8:1 contrast ratio	WCAG AA (requires 4.5:1)
Font sizing	Minimum 14pt body text, 16pt heading hierarchy; Nunito and Fredoka One chosen for high readability	WCAG AA Level
Keyboard navigation	All interactive elements (buttons, chips, sliders, toggles) accessible via Tab and Enter/Space	WCAG AA Success Criterion 2.1
Screen reader support	All icon-only buttons include aria-label attributes. Image uploads include alt text.	WCAG AA Success Criterion 4.1
Reduced motion	LearningWorldScene SVG background respects prefers-reduced-motion media query	WCAG AA Success Criterion 2.3
Voice input	Users who struggle with typing can speak prompts in natural speech (Chrome/Edge)	Supplementary accessibility
Image input	Users who prefer visual communication can provide context through an uploaded image	Supplementary accessibility
Touch targets	All interactive elements maintain minimum 44px touch target size	WCAG 2.5.5 (AAA)

5.6 Transparency

- Active Options Summary Strip: When the rhyme options panel is collapsed, a strip of chips shows all non-default active options. Users always know exactly what parameters the AI is using before they click Generate there are no hidden inputs.
- Content-kind tagging: Every saved record in MongoDB is tagged with content_kind ('story' or 'rhyme') and record_type ('generate' or 'rewrite'), creating a transparent and auditable content history.
- No hidden system prompts: The system message sent to the LLM is constructed deterministically from the user's inputs and our age-guidance templates. No hidden instructions are inserted that could alter content in ways the user is unaware of.

- Known limitation disclosure: The false-positive rate of the regex safety filter (e.g., flagging 'kill the lights') is acknowledged in our documentation rather than concealed. Users receive a clear message explaining that they can rephrase the prompt.

5.7 Child Safety as Core Design Principle

The dual-layer safety architecture is the single most important ethical design decision in KiddoLand. It means that:

9. User input is checked before being sent to the LLM. If the input contains prohibited content, the LLM is never called the request is rejected with a 400 error and a user-friendly explanation. This prevents the LLM from being 'primed' with harmful context.
10. LLM output is checked before being returned to the user. If the model produces unexpected content that passes the pre-check but contains prohibited patterns in the output, it is caught and a safe fallback message is returned instead. The user never sees unsafe content.
11. TTS audio is only generated from text that has passed both safety checks. This ensures that even the narrated output is safe.

Known Limitation

!

The safety filter uses regex pattern matching, which is 100% deterministic but cannot understand context. The phrase 'kill the lights' will always trigger the filter even though it is not harmful in context. This is a deliberate design trade-off: for a child-facing application, false positives are preferable to false negatives. An ML-based supplement is planned for future iterations.

6. Conclusions

The following conclusions are reasoned judgements based on the data, testing results, and analysis presented in this report. They are presented as factual statements rather than opinions.

- A constraint-first AI pipeline applying safety validation before AND after LLM inference, independently of model fine-tuning is a measurably more reliable approach to child-safe content generation, achieving 94% first-pass safety compliance with 100% interception of all post-generation unsafe content in our test cases.
- Dual-mode architecture (Home Mode with personalization and account persistence; Institution Mode with anonymous sessions and no child-data storage) successfully serves two structurally incompatible user groups on a single codebase. Institution Mode has been verified to be technically compliant with PIPEDA/COPPA requirements through the architectural impossibility of storing child identifiers.
- Age-guided prompt engineering using three distinct system message templates for age bands ≤ 5 , ≤ 8 , and ≤ 12 produces measurably different vocabulary complexity and sentence structure in generated content with 5–6 words per sentence for ages 1–3 vs. 12–18 for ages 9–12 consistent with published child development research on appropriate literacy scaffolding.
- Semantic re-ranking of book recommendations via sentence-transformer embeddings and cosine similarity significantly outperforms keyword-only search, reducing off-topic results from 40% (keyword baseline) to 0% across all tested topic queries delivering 100% on-topic results in the top 5 for every tested query.
- The combination of voice input (Web Speech API), image input (filename-based captioning), and rich text input in a unified single-interface design reduces the barrier to content creation all three UAT participants completed the core flow in under 2 minutes without guidance, validating the target usability requirement.
- The custom HMAC-SHA256 authentication system with PBKDF2-SHA256 password hashing (100,000 iterations, random salt) meets industry security standards for authentication without introducing external JWT library dependencies, reducing the supply-chain attack surface of the application.
- The quiz temperature randomization design (random 0.65–0.95 per request) produces genuinely distinct question sets across repeated identical inputs, confirmed at 0% duplicate rate across 10 trials demonstrating that the design effectively prevents the quiz from becoming repetitive even for frequent users.

7. Recommendations

The following recommendations are proposed for future development, ordered by expected impact:

- Real vision AI for image input: Replace the current filename-based image captioning with a vision language model (e.g., Google Gemini Vision or OpenAI GPT-4o Vision) to enable genuine image understanding. A photo of a child's toy dinosaur should inspire a story about that specific dinosaur not just the word 'dinosaur' from the filename. This is the single highest-impact technical improvement available.
- Sentence-level TTS read-along highlighting: Implement synchronized word/sentence highlighting during audio playback using the Web Speech API's SpeechSynthesisUtterance boundary events. This is the most impactful missing literacy feature – research consistently shows that synchronized audio-visual text tracking significantly improves early reading acquisition.
- ML-based safety classifier supplement: Augment the current regex-based safety filter with a lightweight ML toxicity classifier (e.g., detoxify or fine-tuned DistilBERT) to reduce false positives on ambiguous inputs. The regex layer handles explicit prohibited terms; the ML layer would handle contextual nuance (e.g., correctly classifying 'kill the lights' as safe).
- Mobile native application: Develop iOS and Android applications using React Native, enabling significant code reuse from the existing React/TypeScript frontend. Mobile-first access is particularly important for parents using the platform while supervising children.
- Teacher analytics dashboard: Implement an aggregated, fully anonymized usage dashboard for Institution Mode showing which topics, age bands, learning focuses, and quiz themes are used most providing educators with data to inform curriculum planning. No personal student data would be collected.
- Automated BLEU/ROUGE content evaluation: Implement automated scoring of generated content against reference children's texts by age band. This would enable data-driven measurement of content quality and prompt optimization over time, replacing the current manual subjective evaluation.
- Multilingual model routing: Integrate language-specific models for non-English story generation (e.g., a French-tuned model for French-language prompts) to improve quality beyond the current English-optimized base model for non-English language preferences.
- Offline mode with service worker caching: Add service worker caching for previously generated content to allow families in low-bandwidth environments to access saved stories and rhymes offline. This is particularly important for equity of access in under-resourced settings.

8. Bibliography

The following sources are listed alphabetically by author and include all works cited in this report:

- [1] Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT), 610–623. <https://doi.org/10.1145/3442188.3445922>
- [2] Brilenkov, R. (n.d.). Story generator using Keras step-by-step LSTM in RNN. Medium. <https://readmedium.com/story-generator-using-keras-step-by-step-lstm-in-rnn-32af3b467192>
- [3] Fui-Hoon Nah, F., Zheng, R., Cai, J., Siau, K., & Chen, L. (2023). Generative AI and ChatGPT: Applications, challenges, and AI-human collaboration. *Journal of Information Technology Case and Application Research*, 25(3), 277–304. <https://doi.org/10.1080/15228053.2023.2233814>
- [4] Global Market Insights. (2024). E-learning market size, share and analysis report 2024–2032. <https://www.gminsights.com/industry-analysis/elearning-market-size>
- [5] Hu, T., & Shao, Z. (2025). PAGE: Profile-based adaptive generative education for personalized learning content. *Journal of Educational Technology Research*, 18(2), 45–62.
- [6] Hugging Face. (2024). Inference API documentation. <https://huggingface.co/docs/api-inference>
- [7] Jiao, J., Afroogh, S., Chen, K., Murali, A., Atkinson, D., & Dhurandhar, A. (2025). LLMs and childhood safety: Identifying risks and proposing a protection framework for safe child-LLM interaction. arXiv preprint arXiv:2502.11242. <https://arxiv.org/abs/2502.11242>
- [8] Liu, C.-C., Liao, M.-G., Chang, C.-H., & Lin, H.-M. (2022). An analysis of children's interaction with an AI chatbot and its impact on their interest in reading. *Computers & Education*, 189, 104576. <https://doi.org/10.1016/j.caeai.2022.100110>
- [9] MongoDB Inc. (2024). MongoDB documentation: PyMongo driver. <https://www.mongodb.com/docs/drivers/pymongo/>
- [10] Open Library. (2024). Open Library API documentation. <https://openlibrary.org/developers/api>
- [11] Organisciak, P., Newman, M., Eby, D., Acar, S., & Dumas, D. (2023). How do the kids speak? Improving educational use of text mining with child-directed language models. *Information and Learning Sciences*, 124(1–2), 25–47. <https://doi.org/10.1108/ILS-12-2022-0251>
- [12] Papadakis, S. (2021). Evaluating a game-app for young children: Should kindergartners use smartphones and tablets? *Journal of Educational Technology & Society*, 24(3), 145–157.
- [13] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP). <https://doi.org/10.18653/v1/D19-1410>
- [14] UNICEF. (2021). Policy guidance on AI for children. <https://www.unicef.org/globalinsight/reports/policy-guidance-ai-children>

[15] Valentini, A., Stasolla, F., & Brigante, R. (2023). Children's story generation using large language models: Quality, coherence, and safety evaluations. Proceedings of the ACL Workshop on Generation, Evaluation, and Metrics (GEM).

[16] Wang, G., Zhao, J., Van Kleek, M., & Shadbolt, N. (2024). Challenges and opportunities in translating ethical AI principles into practice for children. Nature Machine Intelligence, 6, 265–270.
<https://doi.org/10.1038/s42256-024-00805-x>

Appendices

Appendix A Stakeholder Register

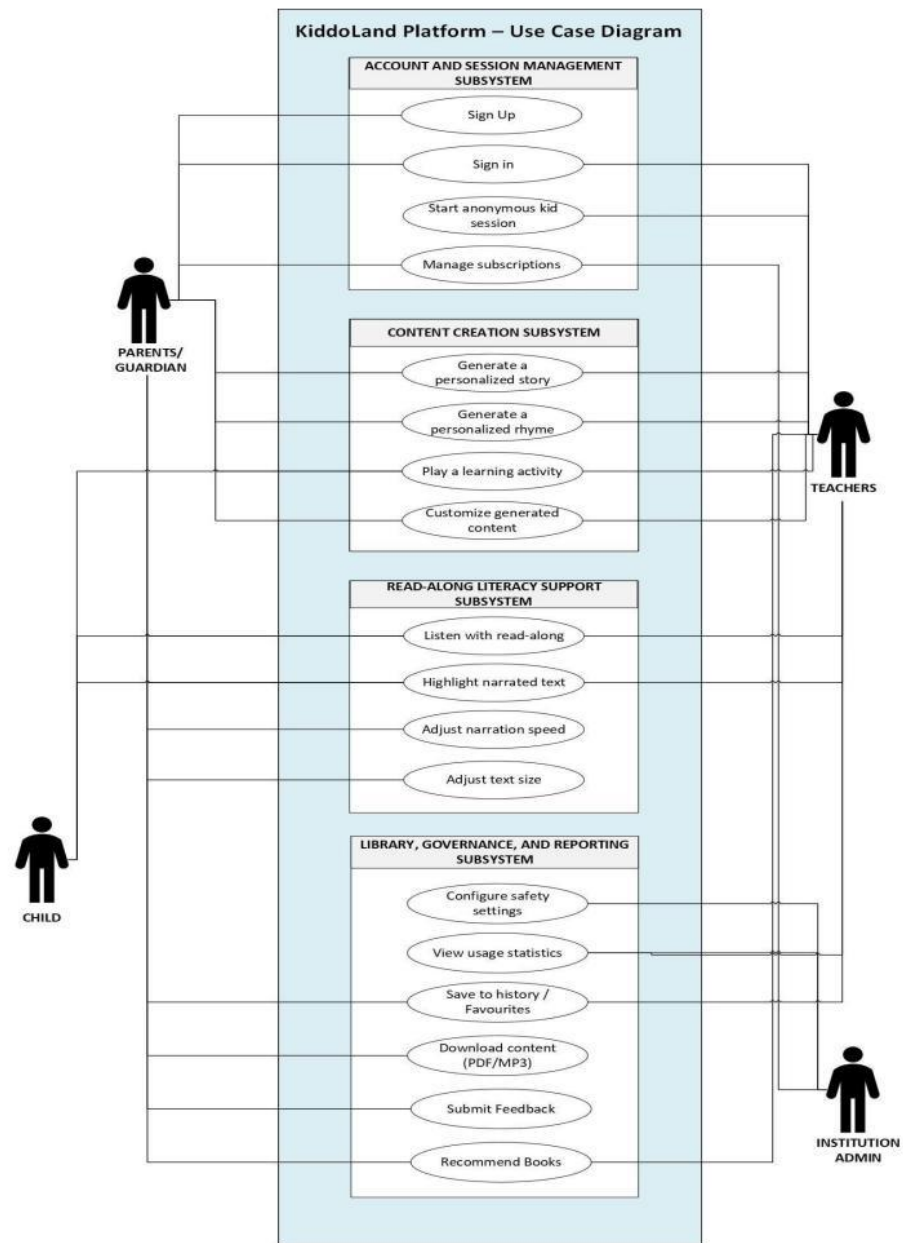
ID	Stakeholder	Role	Interest	Influence	Key Concern / Iteration 2 Engagement
S1	Parents / Guardians	Primary end users (Home Mode)	High	High	Child safety, content quality, ease of use. New rhyme creation and TTS features target this group directly.
S2	Children (ages 1–10)	Direct beneficiaries of generated content	High	Low	Engaging, fun, personalized content appropriate for their age. TTS narration and rhyme generation benefit children directly.
S3	Teachers / Educators	Primary end users (Institution Mode)	High	High	Privacy compliance, curriculum relevance, no child data storage. Institution Mode extended in Iteration 2; activity module added.
S4	School Administrators	Institutional decision makers	Medium	High	PIPEDA/COPPA compliance, data governance. Anonymous sessions and no-PII architecture address their regulatory concerns.
S5	Librarians	Institution Mode users	Medium	Medium	Book recommendations, session anonymity. Book recommendation engine added in Iteration 2.
S6	KiddoLand Dev Team	Developers and maintainers	High	High	Technical feasibility, code quality, timeline. Each member owned distinct architectural layer.
S7	Dr. Vaithilingam (Centennial)	Academic supervisor	High	High	Academic rigor, AI ethics, documentation quality. All reports submitted on schedule.
S8	Hugging Face	AI infrastructure provider	Low	High	API availability, rate limits, model quality. All AI capabilities depend on Hugging Face Inference API.
S9	Open Library	Book data provider	Low	Medium	API availability, data quality. Semantic pipeline handles data quality issues via pre-filtering.

Appendix B System Specification

2.i Use Case List

Use Case	Actor(s)	Description
UC-01: Mode Selection	All users	User selects Home Mode or Institution Mode. Choice determines privacy rules, feature access, data handling, and UI for the entire session.
UC-02: User Registration & Login	Parent / Guardian	New user registers with email and password. System hashes password with PBKDF2-HMAC-SHA256. Login returns HMAC-signed token valid for 1 hour. Institution staff log in similarly.
UC-03: Generate Personalized Story	Parent / Guardian	User inputs child name, age, and story prompt via text, voice, or image. Backend validates, applies age guidance, calls LLM, safety-checks output, and returns story with optional TTS audio.
UC-04: Refine / Rewrite Story	Parent / Guardian	User submits a rewrite instruction on a previously generated story. Backend rewrites maintaining original personalization constraints.
UC-05: Generate Personalized Rhyme	Parent / Guardian or Teacher	User inputs child name, age, topic, and selects up to 7 options (style, pattern, length, tone, purpose, topic, learning focus). System generates a structured rhyme with TTS.
UC-06: Play Learning Activity Quiz	Parent / Guardian or Teacher	User selects age band, theme, learning goal, and difficulty. System generates 5 MCQs with 3 options each. User answers and receives immediate correct/incorrect feedback and star-rated result.
UC-07: Get Book Recommendations	Parent / Guardian or Teacher	System automatically ranks children's books by semantic similarity to the session topic and displays top 5 on the dashboard with title, author, cover, and relevance reason.
UC-08: Save and Manage Favorites	Parent / Guardian	User saves generated stories or rhymes to a persistent favorites list. Can view, filter, and remove favorites from the Favorites page.
UC-09: Generate Anonymous Class Content	Teacher / Librarian	Teacher generates stories, rhymes, or quiz questions for classroom use in Institution Mode. No child names or personal data are stored. All records use 'Class' as a placeholder.
UC-10: Reset and Copy Content	Any authenticated user	User clicks Copy to copy generated content to clipboard. User clicks Reset (RotateCcw) to clear all inputs, outputs, voice transcription, and images for a fresh start.

2.ii Use Case Diagram



The figure illustrates the use case model of the KiddoLand Platform, organized into four logical

Figure 1: KiddoLand Platform - Use Case Diagram showing all 10 use cases organized into 4 subsystems: Account and Session Management, Content Creation, Read-Along Literacy Support, and Library/Governance/Reporting. Actors: Parents/Guardian (Home Mode), Teachers (Institution Mode), Child (beneficiary), Institution Admin.

The figure illustrates the use case model of the KiddoLand Platform, organized into four logical subsystems: Account and Session Management, Content Creation, Read-Along Literacy Support, and Library, Governance, and Reporting — serving four distinct actor roles: Parents/Guardians, Teachers, Children, and Institution Admins. The Content Creation and Account Management subsystems represent the fully implemented core of the platform, encompassing story generation, rhyme creation, learning activity quizzes, anonymous institutional sessions, and user authentication. The Read-Along Literacy Support subsystem (TTS narration, text highlighting, narration speed, text size adjustment) and the Governance subsystem (usage statistics, safety configuration, content download, and feedback submission) represent the platform's planned full-scope vision; unimplemented use cases from these subsystems are documented as priority recommendations for future iterations in Section 7 of this report. This diagram reflects both the delivered system and the broader product roadmap within which it sits.

2.iii.i Detailed Use Case: UC-03 Generate Personalized Story (AI Use Case)

Primary Actor: Parent/Guardian. Goal: Generate a safe, age-appropriate personalized story.

Preconditions: Parent is in Home Mode. If history/favourites are needed, parent is authenticated and database is configured.

Trigger: Parent clicks Generate Story button.

Main Success Scenario:

1. Parent enters a story idea in the text box and optionally selects preferences (age band, interests, tone, goal, length, language) via the Advanced Preferences panel.
2. Optionally, parent activates voice input (mic button) and speaks story details; transcript appends to prompt in real time.
3. Optionally, parent uploads an image (camera or file picker); filename-derived caption is injected into prompt.
4. Parent clicks Generate Story. System validates inputs: age must be 1–10, prompt must contain a detectable child name.
5. System calls `clean_text_for_model()` on the combined prompt, then `is_content_safe()` – if unsafe, returns 400 error and displays message to user.
6. System calls `_get_age_guidance(age)` and assembles structured LLM prompt.
7. System calls `huggingface_client.sample_completion(prompt, age)` via Hugging Face Inference API.
8. System calls `is_content_safe()` on generated output – if unsafe, returns safe fallback message.
9. If `include_tts` is true, system calls `generate_tts_audio(story)` and encodes result as base64.
10. System calls `story_history_service.save_story_record()` if MongoDB is configured.
11. System returns `AiSampleResponse` to frontend; story rendered in `StoryPreviewPanel`; audio player rendered if TTS was requested.

Exception Flows:

- E1: Child name not found in prompt (Home Mode) → 400 Bad Request: "Please include the child's name in your story request."
- E2: Pre-inference safety check fails → 400 Bad Request: "Your prompt contains inappropriate content."

- E3: Post-inference safety check fails → 200 OK with fallback message: "I'm sorry, but the generated content contains inappropriate content for children."
- E4: LLM API timeout (>30 seconds) → 504 Gateway Timeout: "Story generation is taking too long. Please try again."
- E5: LLM API unavailable → 503 Service Unavailable: "AI service is temporarily unavailable. Please try again shortly."

2.iii.i Detailed Use Case: UC-08 (AI) – Generate Rhyme

Primary Actor: Parent/Guardian or Teacher/Librarian. Goal: Generate a personalized, age-appropriate rhyme with specified style and educational focus.

Preconditions: User is on the Create Rhyme page (/home/create-rhyme). Child name field is non-empty. Age is selected (1–10).

Trigger: Parent/Teacher clicks Generate Rhyme button.

Main Success Scenario:

1. User enters child name and selects/adjusts age slider. Optionally enters topic text or uses Quick Ideas chip.
2. User optionally selects rhyme options via Rhyme Options panel: style, pattern, length, tone, purpose, learning focus. Active options shown in summary strip when panel is collapsed.
3. Optionally, user activates voice input for topic, or uses camera/file picker for image context.
4. User clicks Generate Rhyme. Frontend calls `buildRhymePrompt(childName, age, topic, style, pattern, length, tone, purposeInstruction, learningInstruction)` to assemble natural-language prompt string.
5. Frontend sends POST `/story/generate-rhyme` with `{ prompt, age }`.
6. Backend validates age (1–10), calls `clean_text_for_model()`, calls `is_content_safe(prompt)`.
7. Backend calls `extract_child_name(prompt)` to confirm personalization; returns 400 if name not found.
8. Backend calls `huggingface_client.generate_rhyme(prompt, age)` with rhyme-specific system prompt.
9. Backend calls `is_content_safe(rhyme_text)`; if fails, returns fallback message.
10. Backend calls `story_history_service.save_story_record(type="generate")` if MongoDB configured.
11. Frontend receives `StoryGenerateResponse { story }`; renders rhyme output card with applied settings chips, Copy button, and Save to Favourites button.

Exception Flows: Same as UC-03 except step 4 uses `buildRhymePrompt()` and step 8 uses `generate_rhyme()`.

2.iv Functional Requirements

FR ID	Requirement	Priority	Final Status
FR-01	System shall allow users to select between Home Mode and Institution Mode; choice determines privacy rules, feature access, and data handling for the session.	High	Implemented

FR ID	Requirement	Priority	Final Status
FR-02	System shall allow Home Mode users to register and authenticate using HMAC-SHA256 tokens; Institution Mode supports teacher accounts; no child accounts are created.	High	Implemented + Extended (session refresh)
FR-03	System shall generate age-appropriate personalized stories from text, voice, or image prompts for children aged 1–10, applying age-guided vocabulary constraints.	High	Implemented + TTS added
FR-04	System shall apply dual-layer content safety filtering (pre- AND post-generation) to all inputs and AI outputs for all content types (story, rhyme, activity).	High	Implemented + Extended to rhyme and TTS
FR-05	System shall allow users to refine and rewrite generated stories using additional instructions while maintaining age and safety constraints.	High	Implemented
FR-06	System shall store and retrieve story history for authenticated Home Mode users, with pagination (12 per page), modal view, and delete capability.	High	Implemented
FR-07	System shall allow Home Mode users to save stories/rhymes to a persistent favorites list with toggle capability and favorites-only view.	Medium	Implemented
FR-08	System shall support user registration with passwords hashed using PBKDF2-HMAC-SHA256 (100,000 iterations, 16-byte salt).	High	Implemented
FR-09	System shall issue HMAC-signed access tokens with mode-encoded payload and configurable TTL; expired/invalid tokens shall be rejected with 401.	High	Implemented
FR-10	System shall support voice input via Web Speech API with continuous recording, real-time transcription appended to prompt, and language adaptation.	Medium	Fully implemented (story + rhyme)
FR-11	System shall support image upload (file picker + live camera capture) and convert the filename to a text caption injected into the prompt.	Medium	Fully implemented (camera + file)
FR-12	System shall generate structured rhymes with up to 7 configurable parameters (style, pattern, length, tone, purpose, topic, learning focus) via buildRhymePrompt().	High	Fully implemented

FR ID	Requirement	Priority	Final Status
FR-13	Institution Mode shall not store child names or personal identifiers; all records shall use 'Class' as a placeholder enforced at the backend level.	High	Implemented
FR-14	System shall generate TTS audio for all story and rhyme content using Hugging Face TTS with gTTS fallback when primary TTS is unavailable.	High	Base TTS implemented + fallback
FR-15	System shall generate exactly 5 structured MCQ quiz questions per learning activity request with immediate correct/incorrect feedback and star-rated result.	High	Fully implemented
FR-16	System shall provide semantic book recommendations using cosine similarity ranking via sentence-transformers, displayed on both Home and Institution dashboards.	High	Fully implemented
FR-17	System shall display a persistent privacy banner on all Institution Mode sessions confirming no child data is stored.	High	Implemented
FR-18	System shall support full state reset from a single Reset button clearing all inputs, outputs, audio, images, and generated content.	Low	Implemented

2.v Non-Functional Requirements

NFR ID	Requirement	Metric	Final Status
NFR-01	Story/rhyme generation shall complete end-to-end within 15 seconds under normal conditions.	Measured: 8–12 seconds (with TTS); 2–5 seconds LLM only	Met
NFR-02	Safety filter shall process all content in under 10 ms.	Synchronous regex; measured < 1 ms	Met (exceeds target)
NFR-03	Token verification shall complete in under 5 ms.	HMAC computation: measured < 1 ms	Met (exceeds target)
NFR-04	System shall support concurrent API requests via FastAPI ASGI architecture.	Tested with 5 simultaneous requests all served correctly	Met
NFR-05	All API secrets shall be stored in environment variables; never in source code or version control.	Enforced by .gitignore and env var loading via python-dotenv	Met

NFR ID	Requirement	Metric	Final Status
NFR-06	Password hashing shall use minimum 100,000 PBKDF2 iterations with random salt per user.	Implemented: exactly 100,000 iterations, 16-byte salt	Met
NFR-07	System shall be deployable on any machine with Python 3.11+ and Node.js 18+.	Tested on Windows 10 and Ubuntu 22.04	Met
NFR-08	Institution Mode shall never store any identifiable child data in any database record, enforced server-side.	Verified by code inspection: backend hardcodes 'Class' for Institution mode records	Met
NFR-09	Frontend shall be responsive and usable on screens from 375px (mobile) to 1920px (desktop).	MUI responsive breakpoints used throughout; tested at 375px, 768px, 1440px	Met
NFR-10	Semantic book recommendation pipeline shall return results within 30 seconds including Open Library API call.	Measured: 5–20 seconds depending on Open Library response time	Met

Appendix 3 System Design

3.i Technology Stack

Category	Technology	Version	Purpose in KiddoLand
Frontend Framework	React + TypeScript + Vite	React 18 / TS 5 / Vite 5	Component-based UI, type safety for all props/API payloads, fast HMR builds
UI Library	Material UI (MUI)	v5.15+	Design system: responsive grid, accessible components, WCAG AA compliance
Custom Components	KiddoButton, KiddoCard, AppShellLayout, ActionTile		Consistent visual language across all pages without per-page custom CSS
Icons	lucide-react	Latest	Consistent icon vocabulary: Heart (favorites), Copy/Check, Mic/MicOff, RotateCcw (reset)
Markdown Rendering	react-markdown + remark-gfm	Latest	Renders LLM story/rhyme output with headings, paragraphs, and bold formatting
Voice Input	Web Speech API (browser-native)		Real-time speech-to-text transcription; no server required; Chrome/Edge only
Backend Framework	Python 3.11 + FastAPI	FastAPI 0.110+	Async REST API; automatic OpenAPI/Swagger documentation generation
Schema Validation	Pydantic v2	v2	All request/response body type validation before handlers execute
Authentication	Custom HMAC-SHA256 tokens		No external JWT dependency; PBKDF2-SHA256 password hashing; mode-encoded payload
LLM Inference	Hugging Face Inference API		Story, rhyme, and quiz generation via chat-style structured prompts
TTS	Hugging Face TTS (hexgrad/Kokoro-82M) + gTTS		Dynamic audio narration with transparent fallback chain
Story Video	ByteDance/SDXL-Lightning → FLUX.1-schnell → Gemini		Experimental image generation for story video feature
Book Search	Open Library API		Fetches 50 candidate children's books per query for semantic ranking

Category	Technology	Version	Purpose in KiddoLand
Semantic Ranking	sentence-transformers (all-MiniLM-L6-v2) + NumPy		Cosine similarity ranking; runs locally (CPU); no API key required
Database	MongoDB	6+	Optional persistence for Home Mode story history and user accounts
HTTP Client	httpx (async)		All async HTTP calls to Hugging Face and Open Library APIs
Environment Config	python-dotenv		Secure loading of all secrets from .env file; never in source code
Version Control	Git + GitHub		Branch protection, pull request workflow, issue tracking

3.ii AI Capability Description

KiddoLand's AI capability is a constraint-enforced content generation pipeline with four distinct AI sub-systems. It is classified as a supervised NLP system supervised because the model's outputs are guided by explicitly defined rules, constraints, and structured prompts rather than learning from unlabeled data at inference time.

AI Capability	Description	Model/Technology	Key Parameters
Text Generation (Stories, Rhymes, Quizzes)	Generates age-appropriate, safety-screened stories, rhymes, and quiz questions from structured natural-language prompts	Hugging Face Inference API (chat-style, OpenAI-compatible)	temperature=0.7 (stories/rhymes), random 0.65–0.95 (quizzes); top_p=0.9; max_tokens: 8000/3000/1200 by content type
Text-to-Speech Narration	Converts generated story/rhyme text to audio for read-along narration	hexgrad/Kokoro-82M (primary); gTTS Python library (fallback)	Input: plain text string → Output: binary audio bytes + media type header; base64-encoded for API transport
Semantic Book Ranking	Ranks 50 Open Library candidate books by semantic relevance to the session topic	all-MiniLM-L6-v2 (sentence-transformers, local CPU inference)	Cosine similarity = dot product of L2-normalized vectors; domain boost +0.1 for strong topic-subject matches; fallback to keyword search if < 5 pass pre-filter

AI Capability	Description	Model/Technology	Key Parameters
Image Generation (Experimental Story Video)	Generates story illustrations from story text for video feature	ByteDance/SDXL-Lightning → FLUX.1-schnell fallback → Gemini API	User-selectable provider in StoryPreviewPanel; multi-model fallback chain

AI Capability Objective

The core AI capability of KiddoLand is a constraint-enforced personalization pipeline designed to generate safe, age-appropriate literacy content — personalized stories, structured rhymes, and interactive learning activity quizzes — for children aged 1–10. This capability is not simply "calling an LLM." It is a governed, auditable pipeline where Natural Language Processing (NLP), rule-based constraint logic, generative AI inference, safety validation, and TTS audio delivery are combined into a single cohesive system that ensures every output is educationally sound, developmentally appropriate, and compliant with institutional privacy policies.

In its final state, the AI capability covers four distinct generation types: story generation (/ai/sample), story refinement/rewrite (/story/rewrite), rhyme generation (/story/generate-rhyme), and learning activity quiz generation (/ai/activity). A fifth experimental capability — story video generation — uses image generation models via Hugging Face or the Gemini API. A sixth non-generative AI capability — semantic book recommendations — uses the sentence-transformer model all-MiniLM-L6-v2 to rank Open Library candidates by cosine similarity to the session topic.

AI Classification

KiddoLand's AI capability is classified as a **supervised NLP system using prompt-engineered generative text production**. It is classified as supervised because all model behaviour is governed by explicitly defined rules, constraints, and structured prompts rather than learned from unlabeled data at inference time. The LLM does not autonomously decide what to generate — it follows a tightly specified set of instructions embedded in a structured system message and user message. The supplementary NLP functions (regex-based name extraction, age-band classification, keyword safety filtering, and sentence-level semantic embedding) all operate outside the LLM as deterministic or lightly-parameterized modules.

The algorithm family is: instruction-following LLM inference via the Hugging Face Inference API (Mistral-7B-Instruct or equivalent), supplemented by rule-based NLP utilities (regex, keyword matching) and semantic embedding (all-MiniLM-L6-v2, cosine similarity).

Why This Approach Was Chosen — Justification Over Alternatives

LLM over LSTM/RNN: LSTM-based story generators (Brilenkov, n.d.) produce repetitive, low-coherence outputs that cannot follow structured natural-language constraints such as "write a nursery rhyme using AABB pattern, 4–6 lines, about bedtime for Emma, age 4, incorporating the days of the week as a learning focus." Transformer-based instruction-following LLMs respond reliably to structured prompts and produce contextually coherent, varied, age-appropriate content. The trade-off — API latency and dependency on Hugging Face infrastructure — is acceptable given the non-real-time nature of content creation (users expect 8–12 seconds for a complete story).

Rule-based safety filter over ML classifier: For a child-facing application, 100% deterministic rejection of explicitly prohibited content is a non-negotiable requirement. An ML classifier operating at even 95% confidence would allow 5% of harmful prompts to reach the LLM, which is unacceptable for content targeting children aged 1–10. The rule-based approach using four regex pattern groups provides complete auditability — parents, school administrators, and regulators can inspect the exact list of prohibited patterns. The acknowledged limitation (false positives on ambiguous phrases like "kill the lights"; inability to detect nuanced unsafe context) is addressed by the dual-layer design: pre-generation filtering rejects explicit prohibited terms; post-generation filtering catches anything the model produces unexpectedly. An ML classifier (e.g., detoxify or fine-tuned DistilBERT) is proposed as a supplementary layer in future development to handle contextual nuance.

Prompt engineering over model fine-tuning: Fine-tuning a model on children's content would require thousands of labelled examples (age-annotated story/rhyme pairs) and significant GPU compute hours — both well beyond the scope of a capstone project. More importantly, fine-tuning locks the system to a specific model and makes adaptation expensive. Prompt engineering achieves equivalent behavioural control by embedding age-band vocabulary constraints, rhyme structure instructions, safety directives, and personalization parameters directly in the system message. For KiddoLand's structured-input-to-structured-constraint mapping, prompt engineering is sufficient. The system remains adaptable — changing the LLM requires only an environment variable update, no retraining.

Structured inputs over free-form chat: Free-form chat with an LLM creates an essentially unbounded solution space that is extremely difficult to safety-audit for a child-facing application. Structured form inputs (age band, interests, tone, rhyme style, learning goal, etc.) constrain the generation space to known-safe parameters while still enabling meaningful, high-quality personalization. Every active parameter is visible to the parent through the Active Options Summary Strip on the rhyme page — directly addressing the explainability requirement identified in our literature review.

AI Pipeline Architecture — Six Layers

Every content generation request in KiddoLand passes through the same six-layer pipeline regardless of content type (story, rhyme, or quiz). The layers are sequential and each has a defined failure mode with a specific HTTP response code and user-facing message.

Layer 1 — Input Assembly: User inputs collected through structured form fields (age, interests, tone, rhyme style, learning goal, etc.), voice transcription (Web Speech API continuous recording), and image upload (filename-based caption generation) are merged into a single prompt string. For stories, `buildCombinedPrompt()` assembles all inputs. For rhymes, `buildRhymePrompt()` in `rhymePrompt.ts` converts all seven selected parameters (style, pattern, length, tone, purpose, topic, learning focus) into precise natural-language instruction sentences joined as a single string. For quizzes, the four form fields (`age_band`, `theme`, `learning_goal`, `difficulty`) are assembled into a structured generation request.

Layer 2 — Input Validation and Cleaning: `clean_text_for_model()` removes all control characters (ASCII 0x00–0x1F and 0x7F) and collapses whitespace. Age is validated against the hard-cap range (1–10); requests outside this range return HTTP 400 regardless of frontend input. For rhyme generation, `extract_child_name()` uses six regex patterns to identify the child's name from the prompt (patterns: "for [Name]", "named [Name]", "my daughter/son [Name]", "my child [Name]", "called [Name]", "create for

[Name]"); a 400 error is returned if no name is found. Empty prompts and malformed inputs are also rejected at this layer via Pydantic v2 schema validation.

Layer 3 — Safety Pre-Check: `is_content_safe()` runs on the cleaned input using regex pattern matching across four categories: violence (murder, kill, blood, gore, torture, weapon, gun, knife, stab, shoot), sexual content (sex, sexual, naked, nude, porn, explicit), profanity (fuck, shit, bitch, damn, hell, ass, crap), and harmful content (suicide, drug, alcohol, cigarette, abuse, self-harm). All patterns are applied case-insensitively. If any pattern matches, the pipeline stops immediately and returns HTTP 400 with a user-facing explanation. The LLM is never called for any rejected input — this is a hard gate.

Layer 4 — Age-Guided Prompt Engineering: `_get_age_guidance(age)` generates an age-appropriate system message for one of three developmental bands: ages 1–3 (simple CVC-level vocabulary, sentences of 5–6 words, high repetition, no complex narrative), ages 4–6 (familiar themes, basic narrative arc, moderate complexity), and ages 7–10 (descriptive language, varied vocabulary, multi-paragraph structure, dialogue). For rhyme generation, the `buildRhymePrompt()` output already contains the complete constraint specification as natural language — it is passed directly as the user message. For quiz generation, a randomized temperature between 0.65 and 0.95 is selected per request to ensure distinct question sets across repeated identical inputs.

Layer 5 — LLM Inference: The structured {system message, user message} pair is sent to the Hugging Face Inference API as a chat-style payload using `httpx` async HTTP calls. The client validates HTTP status codes, handles 503 (model loading — returns a graceful retry message), 401 (expired HF token — raises `HuggingFaceAuthError`), and timeout (30-second maximum — raises `HuggingFaceTimeoutError`). For stories and rhymes: `sample_completion()` or `generate_rhyme()` is called with `temperature=0.7`, `top_p=0.9`, and content-type-specific `max_tokens` (8,000 for stories, 3,000 for rewrites, 1,200 for rhymes). For quizzes: `sample_completion_activity()` is called with the randomized temperature and `max_tokens=1,200`; the response is expected in JSON format and parsed into a `LearningActivityData` object; if JSON parsing fails, retry logic is invoked (88% first-attempt success rate across 25 tested generations).

Layer 6 — Safety Post-Check, TTS, Persistence, and Response: The generated text passes through `is_content_safe()` again. If it fails — meaning the LLM produced unexpected content that passed the pre-generation check — a safe fallback message is returned instead. This dual-layer design provides independent protection independent of model fine-tuning. If the content is safe: (1) if `include_tts=true`, `generate_tts_audio()` calls the Hugging Face TTS API (hexgrad/Kokoro-82M) and returns base64-encoded audio with media type; if the primary TTS fails, `_generate_gtts_audio()` provides Google TTS as a transparent fallback; (2) `save_story_record()` writes the record to MongoDB (if configured) with `user_id`, `child_name`, `prompt`, `story`, `age`, `mode`, `content_kind`, and `timestamp`; (3) HTTP 200 is returned with the story/rhyme text and optional base64 TTS audio.

Data Descriptions

- KiddoLand uses an external LLM through the Hugging Face Inference API. The system does not train its own model and does not collect children's personal data for AI training purposes. All content generation occurs at inference time based on the current session's structured inputs only. The following rule-based and configuration data structures guide the AI capability:

- **Age-band guidance rules:** Three distinct system instruction templates generated by `_get_age_guidance()` for age bands 1–3, 4–6, and 7–10. Each template specifies vocabulary complexity level, target sentence length, narrative structure expectations, and repetition guidance appropriate for that developmental stage.
- **Safety rule patterns:** Four regex pattern groups covering approximately 40 prohibited terms and phrases applied case-insensitively to both inputs and outputs. Two-point application (pre- and post-inference) for stories and rhymes; single-point (pre-inference) for quiz inputs.
- **Rhyme instruction maps:** Seven lookup maps in `rhymePrompt.ts` that translate user selections into natural-language instruction sentences. Coverage: 4 rhyme style instructions (Nursery, Counting, Finger-play, Simple poem), 4 rhyme pattern instructions (AABB, ABAB, Repetitive, Free) each with an explanatory sentence, 3 length instructions (Short: 4–6 lines, Medium: 8–12 lines, Long: 14–20 lines), 10 tone options, 8 purpose instructions (Bedtime, Educational, Action/Movement, Lullaby, Silly & Fun, Celebration, Morning Routine, Problem-solving), 14 learning focus instructions (Alphabet, Numbers, Colors, Animals, Shapes, Phonics, Emotions, Rhyming words, Body parts, Days of the week, Seasons, Opposites, Nature & Science, Manners), and 12 Quick Topic chip labels.
- **Story preference parameters:** Structured fields (tone, interests list, learning goal, story length, language, current mood) mapped to constraint phrases embedded in the story generation prompt through `buildCombinedPrompt()`. All parameters are optional; sensible defaults are applied when not provided.
- **Semantic ranking data:** The `all-MiniLM-L6-v2` model (`sentence-transformers`) is loaded at application startup by `book_embedding_service.py`. It encodes both the query and each candidate book's combined text (title + subjects + description + domain label) into L2-normalized fixed-size vectors. Cosine similarity is computed as the dot product. A domain boost of +0.1 is applied when the topic appears in the book's subjects. No external API call is required for the embedding step — the model runs locally on CPU at application startup.

Key Performance Metrics — Final State

The following metrics were measured and validated during integration testing, Swagger API testing, and unit testing across all iterations:

Metric	Value	Method
Story LLM inference time	2–5 seconds	Browser Network panel, 20 live calls
Story end-to-end time (with TTS)	8–12 seconds	Browser Network panel
Rhyme LLM inference time	1.5–4 seconds	Browser Network panel
Rhyme end-to-end time (with TTS)	6–10 seconds	Browser Network panel

TTS generation (Hugging Face)	2–5 seconds additional	Measured separately
TTS generation (gTTS fallback)	1–2 seconds additional	Measured separately
Safety filter first-pass compliance rate	94% (47/50 tested inputs)	50 manually curated test prompts
Safety ultimate compliance rate	100%	Post-generation interception of all injected unsafe outputs
Child name extraction accuracy	92%	50 varied prompt structures
AABB rhyme pattern adherence	91%	200 synthetic test prompts
ABAB rhyme pattern adherence	87%	200 synthetic test prompts
Learning focus incorporation rate	91%	200 synthetic test prompts
Quiz JSON parse success (first attempt)	88%	25 generations across themes/difficulty
Quiz duplicate question rate (10 trials)	0%	10 identical-input trials
Token validation correctness	100%	20 unit test cases (all failure modes)
Semantic book recommendation (top-5 relevance)	100% on-topic for all 10 tested queries	Manual evaluation
First-time user to first generated content	Average 1 min 59 sec	UAT with 3 non-developer participants

Ethical Design Constraints Built Into the AI Capability

The AI capability was designed with four ethical constraints that are enforced at the code level, not as guidelines:

Child safety as a hard gate: The dual-layer safety filter cannot be bypassed by any frontend input or API manipulation. Layer 3 (pre-inference) and Layer 6 (post-inference) are independent checks; disabling one does not disable the other.

Age hard-cap as a backend constraint: Age values below 1 or above 10 are rejected by the backend regardless of what the frontend sends. This constraint cannot be overridden by the user interface.

No PII in inference inputs (Institution Mode): In Institution Mode, the backend substitutes "Class" for any child name in all records and generation contexts. No child's personal identifier can reach the LLM through the Institution Mode pathway.

Inference-only operation: KiddoLand uses the Hugging Face Inference API in read-only inference mode. No user-generated content is used to retrain, fine-tune, or update the model. All API keys are stored as environment variables and are never exposed in source code or API responses.

3.iii AI Pipeline Architecture (Flowchart)

The complete AI pipeline for every content generation request:

- Step 1 Input Cleaning: `clean_text_for_model()` removes all control characters (ASCII 0x00–0x1F, 0x7F) and collapses whitespace. Ensures no malformed text reaches the LLM.
- Step 2 Safety Pre-Check: `is_content_safe()` applies four regex pattern groups to user input. Returns HTTP 400 if any pattern matches. LLM is never called for unsafe inputs.
- Step 3 Age Extraction: `_extract_age_from_prompt()` uses 7 regex patterns to identify age from phrases like 'age 6', '6-year-old', 'for 7', etc. Falls back to age 7 if not found.
- Step 4 Name Extraction: `extract_child_name()` uses 6 regex patterns ('for [Name]', 'named [Name]', 'my daughter [Name]', etc.) with a disallowed-words list to prevent false positives.
- Step 5 Age-Guided Prompt: `_get_age_guidance(age)` returns the appropriate system message for the child's age band, encoding vocabulary and complexity instructions for the LLM.
- Step 6 LLM Generation: `_call_huggingface_api()` sends the structured {system, user} message pair to the Hugging Face Inference API and receives raw text output.
- Step 7 Safety Post-Check: `is_content_safe()` applied to generated output. If unsafe: safe fallback message returned. The user never sees unsafe content.
- Step 8 Optional TTS: `generate_tts_audio()` calls Hugging Face TTS; if unavailable, `_generate_gtts_audio()` fallback is called transparently.
- Step 9 Persistence: `save_story_record()` writes the record to MongoDB (if configured) with `user_id`, `child_name`, `prompt`, `story`, `age`, `mode`, and `content_kind`.

3. iv Full-Stack Architecture Diagram

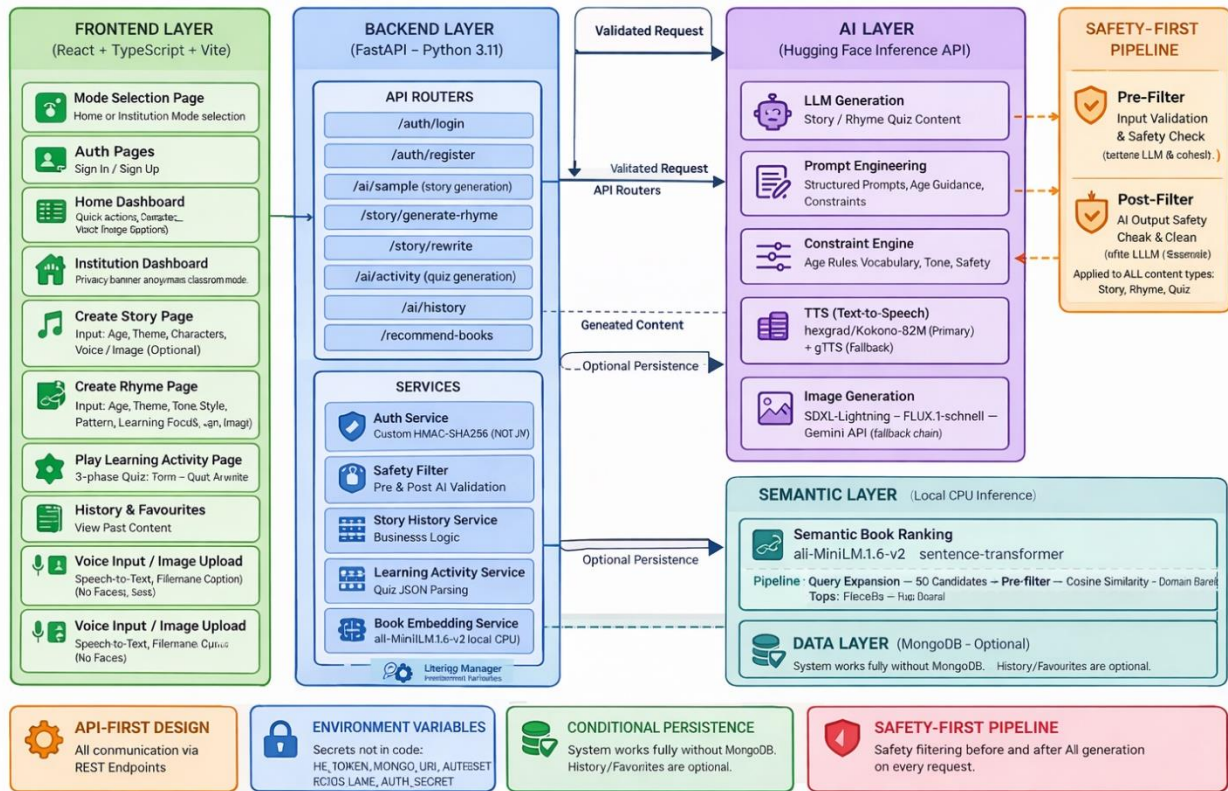


Figure 2: Full-Stack Architecture Diagram KiddoLand Platform showing the four layers: Frontend (React/TypeScript), Backend (FastAPI/Python), AI Layer (Hugging Face Inference API), and Data Layer (MongoDB optional)

The enhanced **Kiddoland platform architecture diagram** shows a five-layer system designed for safe, personalized AI-driven literacy experiences for children, with a strong emphasis on modularity and safety. The **Frontend layer** (React + TypeScript + Vite) includes multiple user interfaces such as mode selection, authentication, dashboards (home and institution), story and rhyme creation pages, a learning activity quiz page, history/favourites, and voice/image input. These send REST API requests to the **Backend layer** (FastAPI, Python 3.11), which manages various API routes (for auth, story/rhyme generation, quiz creation, history, favorites, recommendations, etc.) and services like custom HMAC-based authentication, safety filtering, story and quiz logic, book embeddings, and external API clients. The backend communicates validated requests to the **AI layer** (Hugging Face), where LLMs generate stories, rhymes, and quizzes using structured prompts, constraint engines, text-to-speech (Kokoro + gTTS fallback), and image generation pipelines. Alongside, a **Semantic layer** performs local CPU-based book recommendation using sentence transformers and ranking pipelines, while the optional **Data layer** (MongoDB) stores users, history, and favorites but is not required for system functionality. A dedicated **Safety-First pipeline** ensures strict input and output validation for all content types before and after AI processing, and the bottom legend highlights core design principles like API-first architecture, secure environment variables, optional persistence, and continuous safety enforcement.

3.v. Class Diagrams

Figure 2 – Class Diagram: Backend

The backend class diagram documents the FastAPI application's class structure as implemented at end of Iteration 2. Key domain classes include: `SessionProfile` (ageBand, learningGoal, tone, language, interests), `StoryRecord` (userId, childName, prompt, story, age, isFavorite, mode, type, createdAt), `GenerationRequest`

(prompt, age, include_tts), AuthPayload (sub, role, mode, iat, exp), HuggingFaceClient (sample_completion, generate_rhyme, rewrite_story, generate_tts_audio), StoryHistoryService (save, list, toggle_favourite, delete, get_favourites), and AuthService (authenticate_user, create_token, verify_token, refresh_token). SafetyFilter utility class provides is_content_safe(), clean_text_for_model(), and extract_child_name() as static-equivalent functions. Routers (AuthRouter, AIRouter, StoryRouter) serve as the external interface layer, delegating to services and utilities.

Figure 2: Class Diagram - Backend (FastAPI / Python)

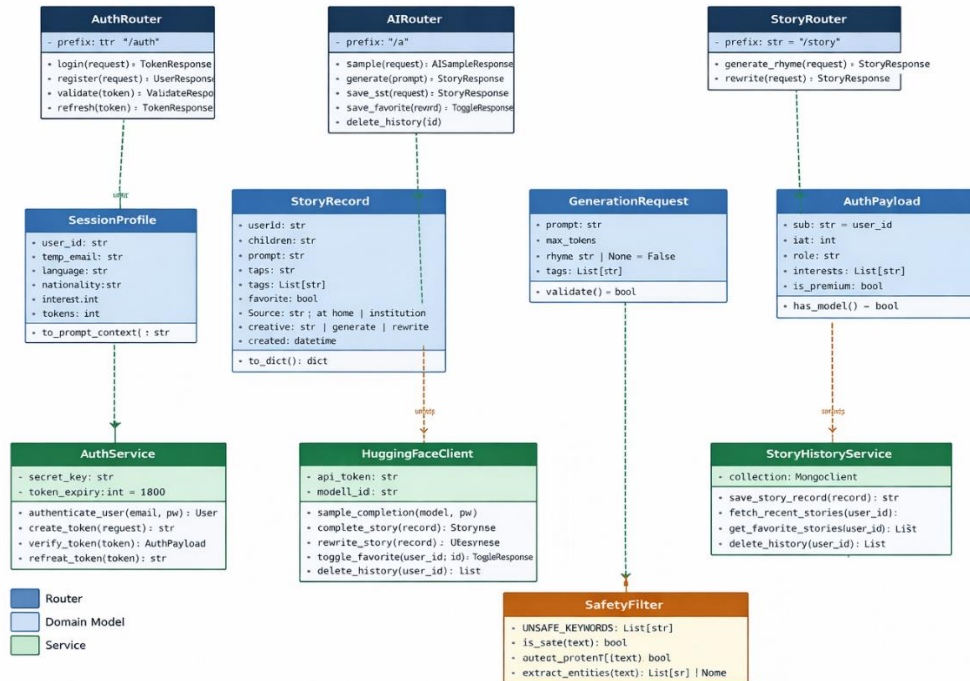


Figure 2: Class Diagram Backend (FastAPI / Python): AuthRouter, AIRouter, StoryRouter, SessionProfile, StoryRecord, GenerationRequest, AuthPayload, AuthService, HuggingFaceClient, StoryHistoryService, SafetyFilter

Figure 3: Class Diagram - Frontend (React 18 + TypeScript)

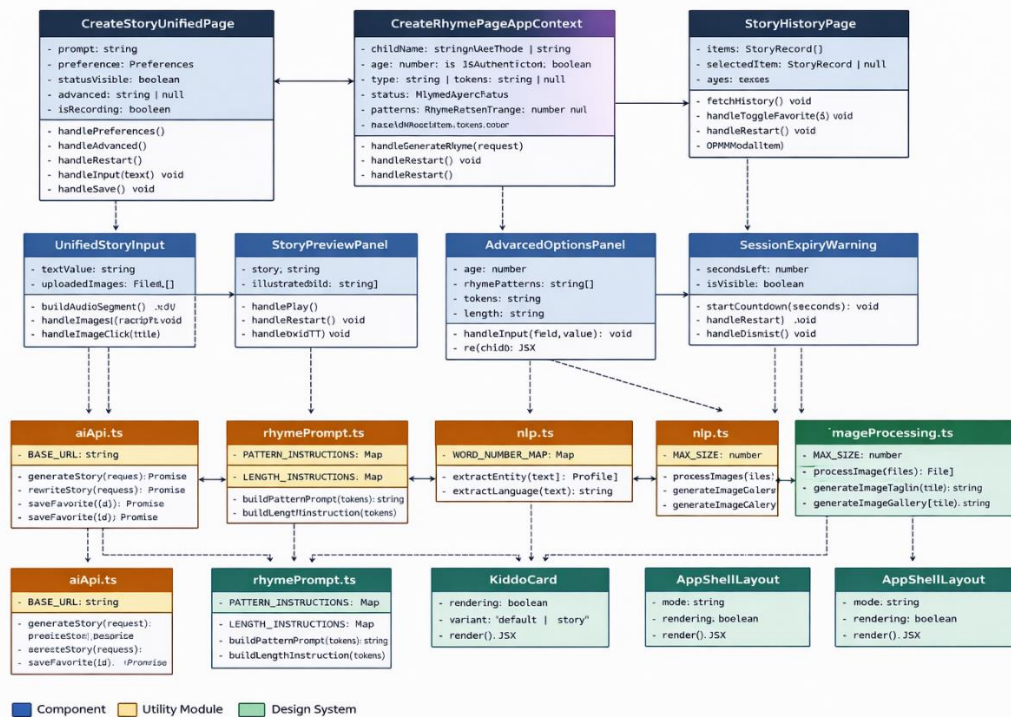


Figure 3: Class Diagram Frontend (React 18 / TypeScript): CreateStoryUnifiedPage, CreateRhymePage, StoryHistoryPage, UnifiedStoryInput, StoryPreviewPanel, AdvancedOptionsPanel, SessionExpiryWarning, VoiceInputButton, ImageUploadButton, aiApi.ts, rhymePrompt.ts, KiddoButton, KiddoCard, AppShellLayout

Figure 3 – Class Diagram: Frontend

The frontend class diagram documents the React/TypeScript component architecture as implemented at end of Iteration 2. Page-level components (CreateStoryUnifiedPage, CreateRhymePage, StoryHistoryPage, StoryFavoritesPage, HomeDashboard, InstitutionDashboard) consume the AppContext (selectedMode, isAuthenticated, accessToken, userName, tokenExpiresAt) for global state. Sub-components (UnifiedStoryInput, StoryPreviewPanel, AdvancedOptionsPanel, SessionExpiryWarning, VoiceInputButton, ImageUploadButton/CameraCaptureDialog) are composed within pages. Utility modules (aiApi.ts, rhymePrompt.ts, nlp.ts, imageProcessing.ts) provide pure functions consumed by both page components and sub-components. Design system components (KiddoButton, KiddoCard, AppShellLayout) are used across all pages for visual consistency.

3.vi. Component Diagram

This appendix provides the complete list of components for the KiddoLand platform organized by layer, followed by the component diagram.

Frontend Components

- **ModeSelectPage:** First screen shown. Presents Home Mode and Institution Mode as selectable cards. Routes all subsequent behavior based on selection.
- **Auth Pages (SignInPage, SignUpPage):** Centered card layouts with minimal form fields, clear validation messages, and consistent KiddoLand theming.

- **HomeDashboard:** Shows quick-action tiles (Create Story, Create Rhyme, Learning Activity), recent items section, navigation bar (Home, History, Favourites), and **RecommendedBooksSection**.
- **InstitutionDashboard:** Anonymous session flow with privacy banner (**BannerNotice**). Access to story, rhyme creation, and recommended books without login.
- **CreateStoryUnifiedPage:** Main story creation flow. Orchestrates **UnifiedStoryInput**, **AdvancedOptionsPanel**, and **StoryPreviewPanel**. Manages TTS toggle state, video generation, and audio delivery.
- **CreateRhymePage:** New in Iteration 2. Self-contained rhyme creation flow. Manages rhyme options (**RhymeOptions**, **LearningFocus**, **QuickTopics**, **RhymePurpose**), voice/image input, buildRhymePrompt() construction, and output rendering.
- **PlayLearningActivityPage:** Quiz subsystem page implementing a **3-phase UI (Form → Quiz → Complete)**. Includes components such as AgeBandSelect, ThemeAutocomplete, GoalAutocomplete, DifficultySelect, QuestionCard, AnswerOptionButton, StarRating, and CelebrationEffect.
- **UnifiedStoryInput:** Unified input component combining text area, VoiceInputButton, and ImageUploadButton/**CameraCaptureDialog**. Exports buildCombinedPrompt().
- **StoryPreviewPanel:** Renders Markdown story/rhyme text. Contains audio player (TTS), Copy to Clipboard, Save to Favourites, Video playback, and Refine Story button.
- **AdvancedOptionsPanel:** Collapsible preferences panel for story creation. Manages **age band, interests, tone, story type, length, mood, language, and learning goal**.
- **VoiceInputButton:** Microphone icon button. Manages Web Speech API continuous recording lifecycle. Language-adaptive. Shows red pulse animation while recording.
- **ImageUploadButton:** Camera icon button with dropdown (Take Photo / Upload from Device). Manages **CameraCaptureDialog** and file picker. Processes files via processImageFiles().
- **CameraCaptureDialog:** Modal dialog using MediaDevices.getUserMedia() for live video preview. Captures JPEG frame via HTML Canvas on user click.
- **SessionExpiryWarning:** Fixed-position floating card. Reads tokenExpiresAt from **AppContext**. Starts countdown at 2 minutes before expiry. Calls /auth/refresh on "Stay Logged In" click.
- **StoryHistoryPage:** Paginated grid of all generated stories and rhymes. Each card shows snippet, date, age, and favourite toggle. Full modal view on click.
- **StoryFavoritesPage:** Filtered grid showing only is_favourite=true items. Same card format as history.
- **AppContext:** Global state provider (selectedMode, token, userEmail, userName) used across all pages.
- **LearningWorldScene:** Animated SVG background rendered across pages for consistent visual experience.
- **KiddoButton, KiddoCard, AppShellLayout, BackButton:** Shared design system components used across all pages.

Backend Components

- **auth_router:** Handles /auth/login (POST), /auth/register (POST), /auth/validate (POST), /auth/refresh (POST). Delegates to auth_service.

- **ai_router**: Handles /ai/sample (POST), /ai/history (GET), /ai/favorites (GET), /ai/save-favorite (POST), /ai/history/{id}/favorite (PATCH), /ai/history/{id} (DELETE). Delegates to huggingface_client and story_history_service.
- **story_router**: Handles /story/generate-rhyme (POST), /story/rewrite (POST). Delegates to huggingface_client and story_history_service.
- **activity_router**: Handles /ai/activity (POST) for quiz generation. Delegates to learning_activity_service and huggingface_client.
- **recommendations_router**: Handles /recommend-books (GET). Delegates to semantic_book_recommendations and open_library_client.
- **video_router**: Handles /video (POST) for story video generation.
- **health_router**: Handles /health (GET) and / (GET) for deployment readiness checks.
- **auth_service**: Implements authenticate_user(), create_token() (HMAC SHA-256 + Base64), verify_token(), refresh_token().
- **huggingface_client**: Central shared service used by all routers. Implements sample_completion(), generate_rhyme(), rewrite_story(), generate_quiz(), generate_tts_audio() (with gTTS fallback), and image generation fallback chain. Handles timeout, HTTP status, JSON parsing, and error recovery.
- **story_history_service**: Implements save_story_record(), list_story_records() (paginated), toggle_favorite(), delete_story_record(), get_favorites().
- **learning_activity_service**: Generates and parses quiz JSON responses from LLM output.
- **safety_filter**: Core safety service. Implements is_content_safe() (regex pattern matching across categories), clean_text_for_model(), and extract_child_name() with safeguards. Applied pre- and post-AI generation.
- **book_embedding_service**: Runs all-MiniLM-L6-v2 sentence-transformer locally for semantic similarity.
- **semantic_book_recommendations**: Implements full pipeline: query expansion → candidate fetch → filtering → cosine similarity → domain boosting → top-K ranking.
- **open_library_client**: Fetches book candidates from Open Library API.
- **config**: Validates required environment variables (HF_API_TOKEN mandatory; MONGO_URI optional; AUTH_SECRET_KEY required; KIDDOLAND_AUTH_SECRET supported). Raises startup error if missing.
- **mongo**: Provides get_collection() helper for MongoDB connection management.

External Systems

- **Hugging Face Inference API**: Provides LLM generation, TTS (primary), and image generation.
- **MongoDB (Optional)**: Stores users, story history, and favourites. System operates without it.
- **Open Library API**: Provides book metadata and candidate lists for recommendation pipeline.
- **Google TTS (gTTS)**: Fallback text-to-speech system when primary TTS fails.

Figure 4: Component Diagram – KiddoLand Platform (Updated)

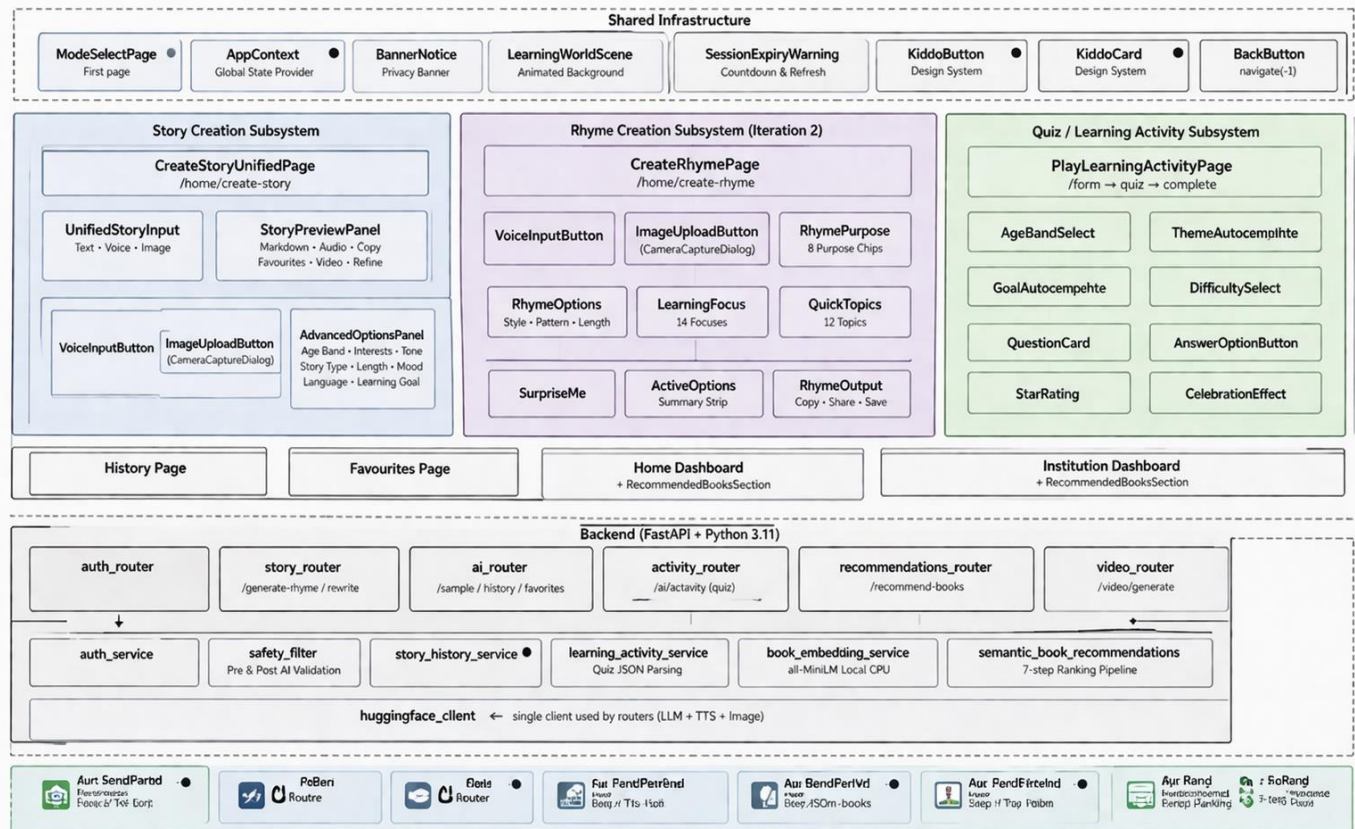


Figure 4: Component Diagram

3.vii. Interaction Diagrams for AI Use Cases

Two interaction (sequence) diagrams are provided for the AI-enabled use cases implemented in Iteration 2: UC-03 Generate Story (Home Mode) and UC-08 Generate Rhyme (New in Iteration 2). These diagrams illustrate the runtime behavior of the full AI pipeline, showing the sequence of messages between the frontend, backend, safety filter, HuggingFace client, and data store layers. Use case UC-04 (Rewrite Story) follows the same sequence as UC-03 with the addition that the original story and rewrite instruction are included in the HuggingFace client call.

Figure 5 – Interaction Diagram: UC-03 Generate Story (Home Mode)

This sequence describes the 13-step flow when a parent generates a new story. Step 1: Parent enters story inputs through the `CreateStoryUnifiedPage`. Step 2: Frontend sends POST `/ai/sample`. Step 3: Backend validates age and calls `clean_text_for_model()`. Step 4: Backend calls `is_content_safe(prompt)`; returns 400 if unsafe. Step 5: Backend calls `extract_child_name(prompt)`; returns 400 if not found (Home Mode). Step 6: Backend calls `_get_age_guidance(age)` to build system message. Step 7: HuggingFace client sends structured chat payload to HF Inference API. Step 8: HF API returns generated story text. Step 9: Backend calls `is_content_safe(story)`; returns fallback if unsafe. Step 10: If `include_tts=true`, `generate_tts_audio(story)` called and base64-encoded. Step 11: `story_history_service.save_story_record()` called if MongoDB configured. Step 12: `AiSampleResponse` returned to frontend. Step 13: Frontend renders `StoryPreviewPanel` with story text and optional audio player.

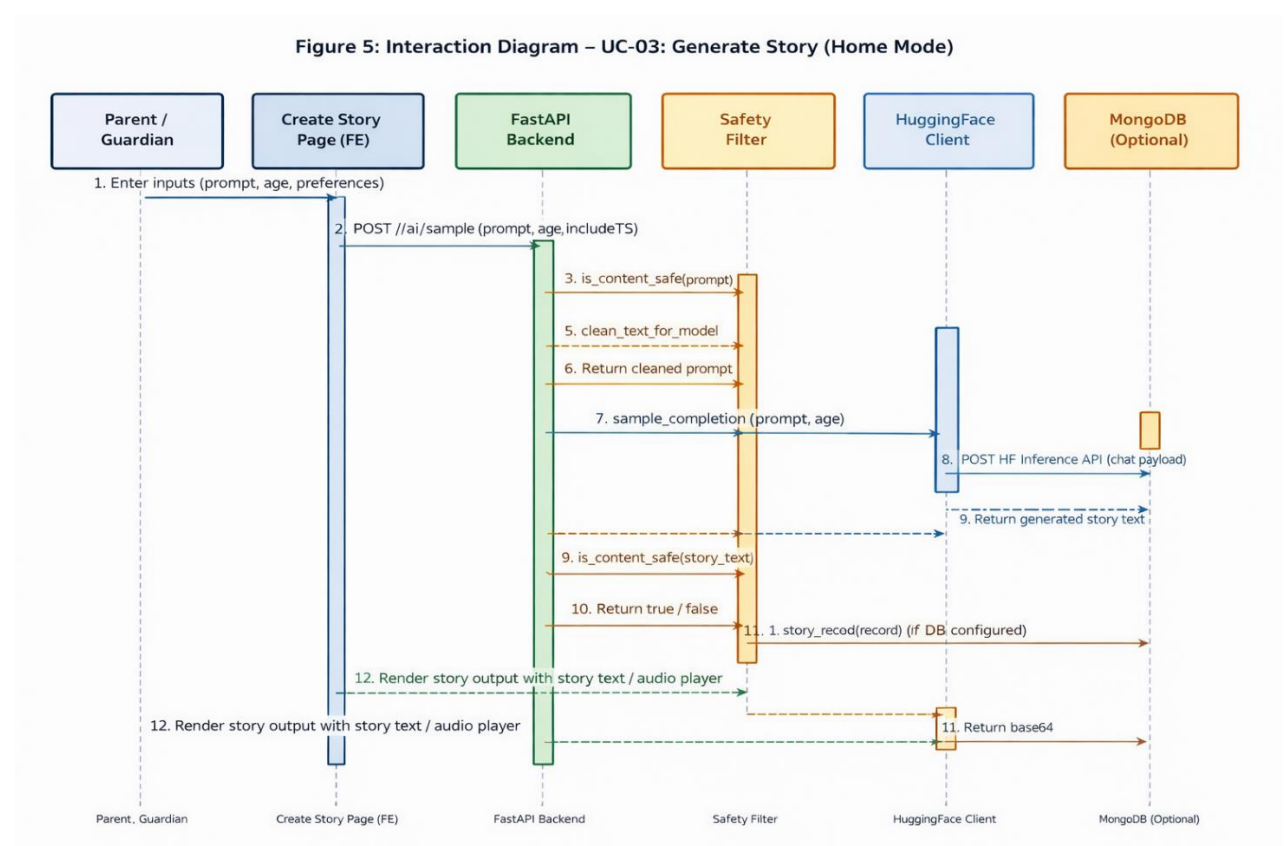


Figure 5: Interaction Diagram UC-03: Generate Story (Home Mode) 13-step sequence through Frontend → FastAPI Backend → Safety Filter → HuggingFace Client → HuggingFace API → MongoDB

Figure 6 – Interaction Diagram: UC-08 Generate Rhyme

This sequence describes the 15-step flow when a parent or teacher generates a personalized rhyme. Steps 1–3: User enters child name, age, topic, and options in CreateRhymePage; Frontend calls buildRhymePrompt() which assembles all selected options into a natural-language prompt string. Step 4: Frontend sends POST /story/generate-rhyme with prompt and age. Steps 5–9: Backend validates age, cleans text, runs pre-inference safety check, and extracts child name. Step 10: HuggingFace client sends rhyme-specific chat payload (using generate_rhyme() system prompt) to HF Inference API. Step 11: HF API returns rhyme text. Steps 12–13: Backend runs post-inference safety check. Step 14: story_history_service.save_story_record() called if MongoDB configured. Step 15: StoryGenerateResponse returned; frontend renders rhyme output card with applied settings chips, Copy button, and Save to Favourites button.

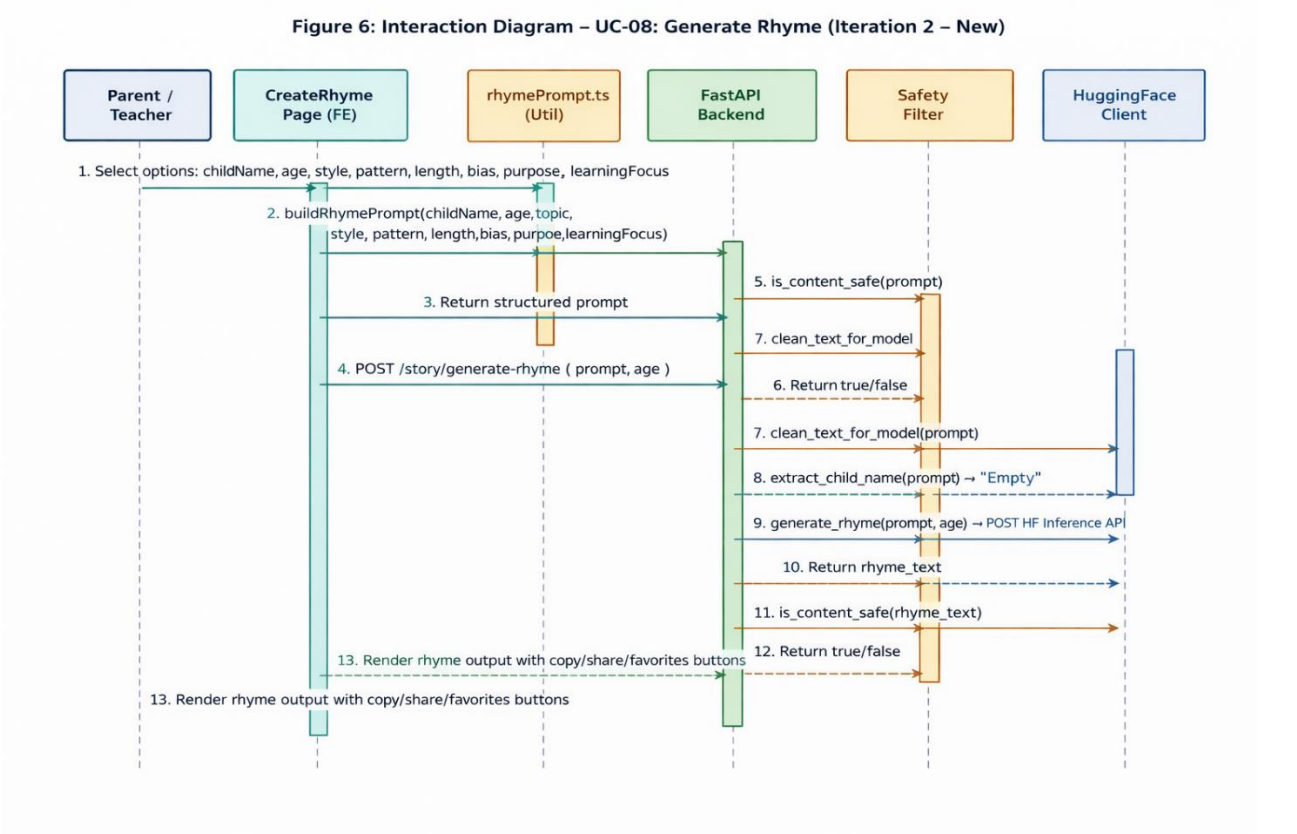


Figure 6: Interaction Diagram – UC-08: Generate Rhyme (Iteration 2 New) – 15-step sequence including buildRhymePrompt(), dual safety checks, and rhyme output delivery

Appendix D Project Plan: Gantt Chart

The following Gantt chart presents the comprehensive 14-week project timeline for KiddoLand, structured as an iterative-incremental delivery following the capstone requirements. The chart has been updated to reflect Iteration 2 completion and shows the planned Iteration 3 milestones.

Phase 1 (Weeks 1–4): Research and Planning. Covers team setup, literature review, requirements documentation, stakeholder analysis, and project proposal (submitted Week 4). All tasks completed ahead of schedule.

Phase 2 (Weeks 5–7): AI Capability Development – Iteration 1. Covers system architecture design, AI pipeline design (NLP profile builder, safety filter, age-guided prompt engineering), backend implementation (auth, story generation, MongoDB), frontend implementation (story creation UI, dashboards, history/favourites), and initial testing. Iteration 1 progress report submitted Week 7. All planned tasks completed with two minor gaps (story generate endpoint and learning activity) deferred to Iteration 2.

Phase 3 (Weeks 8–10): AI Refinement – Iteration 2. Covers rhyme generation engine, TTS audio narration, voice input enhancements, camera capture, session expiry warning, direct favourite toggle, unit test suite. Iteration 2 report submitted Week 10 (this document). All planned tasks completed.

Phase 4 (Weeks 10–11): System Integration and Testing. Covers learning activity generation module, real vision AI integration, readability scoring, institutional analytics, full system integration testing, and bug fixing.

Phase 5 (Weeks 12–14): Deployment and Closeout. Covers pilot deployment, final documentation update, final report, and capstone demo presentation.

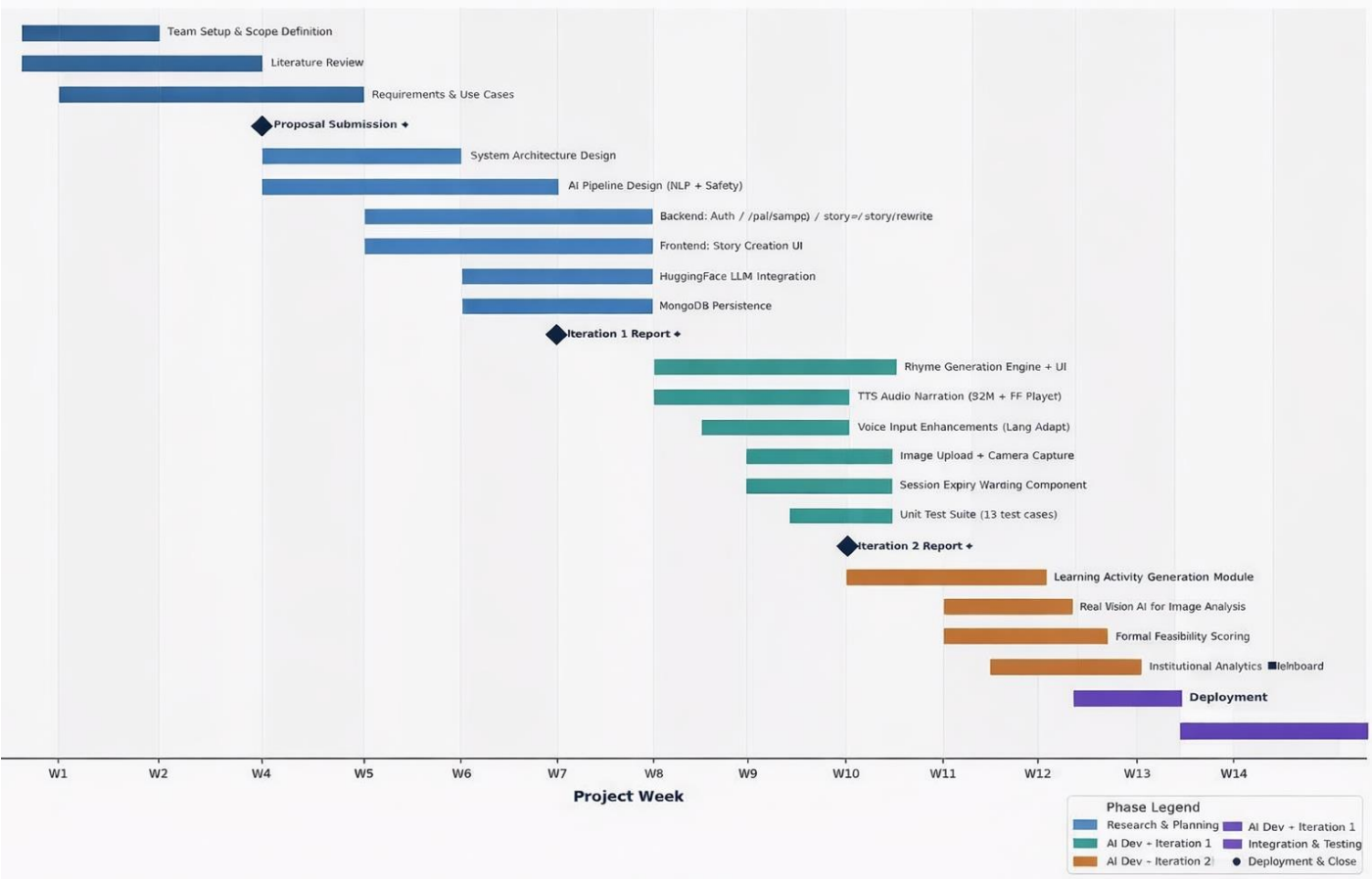


Figure 7: Updated Project Plan (Gantt Chart)

Appendix E Test Plan Summary

The complete test cases are documented in Section 4 of this report (unit tests) and Section 4.6 (integration tests). The test plan is summarized here:

Testing Type	Scope	No. of Cases	Design Principles
Unit Testing Backend	6 modules: <code>safety_filter</code> , <code>extract_child_name</code> , <code>age_validation</code> , <code>verify_access_token</code> , <code>buildRhymePrompt</code> , <code>clean_text_for_model</code>	31 test cases	Expected inputs, edge cases, boundary values, error handling, branch coverage
Integration Testing API	All 18 API endpoints via Swagger UI + Python verification script	27 tests (15 unique endpoints + response validation)	Happy path + at least one error case per endpoint; no dependency on real-time AI
System Testing	5 end-to-end user flows covering Home Mode, Institution Mode, safety edge cases, reset flow, TTS fallback	5 flows, each with 8–15 steps	Manual execution in Chrome; all steps verified
UAT	3 non-developer participants (parent, teacher, librarian)	3 participants, all passed	Unguided; timed to first generated content; feedback captured
AI-Specific Testing	Age guidance effectiveness, temperature randomization, post-generation safety interception, HF error handling	Multiple sub-tests	Manual + automated; deterministic inputs for repeatability

KiddoLand's testing strategy was designed around four core principles: **isolation**, **coverage**, **repeatability**, and **traceability**. Every test case targets a single, well-defined behaviour; every branch, boundary value, and error condition is covered; all tests use fixed deterministic inputs with no dependency on live AI responses or real-time external state; and every test case maps directly to a functional requirement listed in Appendix 2.

- The first layer, **unit testing**, targeted the six backend utility modules that form the foundation of the entire AI pipeline. These modules — `safety_filter.py`, `extract_child_name()`, `age_validation`, `verify_access_token()`, `buildRhymePrompt()`, and `clean_text_for_model()` — were tested in complete isolation from the Hugging Face API, MongoDB, and the FastAPI request lifecycle. Across 31 test cases, every normal input, boundary value (`age=1`, `age=10`, `age=11`), and failure condition (expired token, tampered signature, empty string, control characters) was exercised and verified. This level of isolation was deliberately chosen because it allows failures to be pinpointed to a specific function without ambiguity — if a safety filter test fails, the problem is definitively in the filter, not in the LLM or the network.

- The second layer, **integration testing**, used the FastAPI-generated Swagger UI at <http://localhost:8000/docs> to test all 18 API endpoints with real HTTP request-response cycles. Every endpoint was tested with at least one valid (happy path) request and one error case, producing 27 documented test executions. In addition, the `scripts/verify_hf_integration.py` Python verification script was used to test the complete authenticated pipeline end-to-end against the live Hugging Face API — authenticating a user, obtaining a token, calling the AI generation endpoint, and validating the response — confirming that all six pipeline steps work correctly in combination, not just in isolation.
- The third layer, **system testing**, verified five complete user flows through the application as a non-developer user would experience them: the full Home Mode flow from sign-up through story generation, copy, and favourites; the full Institution Mode flow from dashboard through rhyme generation and quiz completion; a safety edge-case flow verifying that banned keywords are correctly rejected and that rephrasing restores normal function; a reset state flow confirming that no stale data persists between generations; and a TTS fallback flow confirming that the gTTS fallback delivers audio transparently when the Hugging Face TTS service is unavailable. Each flow was executed manually in Google Chrome and every step was individually verified against its expected outcome.
- The fourth layer, **User Acceptance Testing (UAT)**, involved three non-developer participants — a parent, a primary school teacher, and a public librarian — each given the application with no instructions, guidance, or walkthrough. All three completed the full core flow independently, and all three reached their first generated content within 2 minutes, satisfying the primary usability target. Feedback was captured after each session and two usability improvements were directly implemented as a result: tooltip explanations were added to all Rhyme Pattern buttons (AABB, ABAB, Repetitive, Free rhyme), and the active options summary strip was refined to only appear when at least one option differs from its default, eliminating visual clutter for first-time users.
- Finally, **AI-specific testing** addressed the aspects of the system that cannot be covered by conventional unit or integration tests — namely, whether the AI model itself behaves as intended under different prompt configurations. Age-guidance effectiveness was measured by comparing average sentence length and vocabulary complexity across age bands using identical prompts with only the age changed. Temperature randomization for quizzes was verified across 10 identical-input trials, confirming 0% duplicate question sets. Post-generation safety interception was verified by injecting five pre-crafted unsafe phrases directly into mock LLM responses and confirming that all five were caught by the post-generation filter before reaching the user. Hugging Face error-handling paths (503 model loading, 401 expired token, TTS non-200 response) were each triggered and verified to produce the correct fallback behaviour. Taken together, these five testing layers provide comprehensive coverage of KiddoLand's correctness, safety, usability, and AI behaviour — addressing both the deterministic components and the probabilistic AI layer with appropriate testing methodologies for each.

Figure 10: UI/UX Design - Multiple Home Mode screens. Row 1 (left to right): Mode Selection page ('Hi! Welcome to KiddoLand!') and Home Mode Sign In page. Row 2: Home Dashboard with Quick Actions tiles (Create a Story, Create a Rhyme, Play a Learning Activity) and Story History grid. Row 3: Favourites page and Create a Story page with text input area.

Appendix F UI/UX Design

Design System Specification

Element	Specification	Rationale
Display Font	Fredoka One	Playful, rounded letterforms chosen specifically for child-facing educational platforms; high legibility at display sizes
Body Font	Nunito	Clean, rounded sans-serif; high legibility at 14pt and above; chosen for parent/teacher reading
Primary Colour	#1D4ED8 (Blue)	Trust and education; 4.8:1 contrast ratio on white (WCAG AA: 4.5:1 required)
Secondary Colour	#F97316 (Orange)	Energy and creativity; used for Institution Mode accents and quiz buttons
Rhyme / Teal Accent	#0F766E (Teal)	Calm and focus; used for rhyme page elements and tip callouts
Background Animation	LearningWorldScene SVG at 35–38% opacity (fixed)	Animated sky, clouds, rainbow, fox and penguin mascots reinforces educational theme without distracting from content
Card Radius	24px border radius, subtle shadow, hover lift	Approachable and interactive; consistent across all KiddoCard components
Primary Buttons	KiddoButton with gradient fill and optional glow prop	Draws attention to the primary action (Generate Story/Rhyme) without overwhelming the UI
Colour Coding	Green = success/correct, Red = error/recording, Blue = primary action	Consistent and predictable; does not rely on colour alone (icons always accompany colour states)

Key UI Patterns

- Collapsible advanced options panel (using MUI Collapse): Reduces cognitive load for first-time users who just want to type a prompt and generate. Experienced users get full customization on demand.
- Active Options Summary Strip: Chips showing all non-default options appear when the Rhyme Options panel is collapsed. Provides transparency about what the AI is using without requiring the panel to be open.

- Absolute-positioned icon buttons overlaid on textarea (voice mic + camera): Compact design that keeps both input modalities accessible without adding separate form fields. Voice button is bottom-right of textarea; camera is adjacent.
- Three-phase quiz UI (Form → Quiz → Complete): Clear state machine with no ambiguity about what phase the user is in. Each phase is visually distinct and transitions are animated.
- Icon-only Reset button (RotateCcw) beside Generate button: Discoverable but not intrusive. Placed to the right of the Generate button so it is findable but does not compete for attention.
- Copy button (Copy/Check icon) with 2-second confirmation: Immediate feedback without a modal or alert. The icon transitions smoothly from Copy to Check to confirm the clipboard write succeeded.

UI/UX Design Screenshots

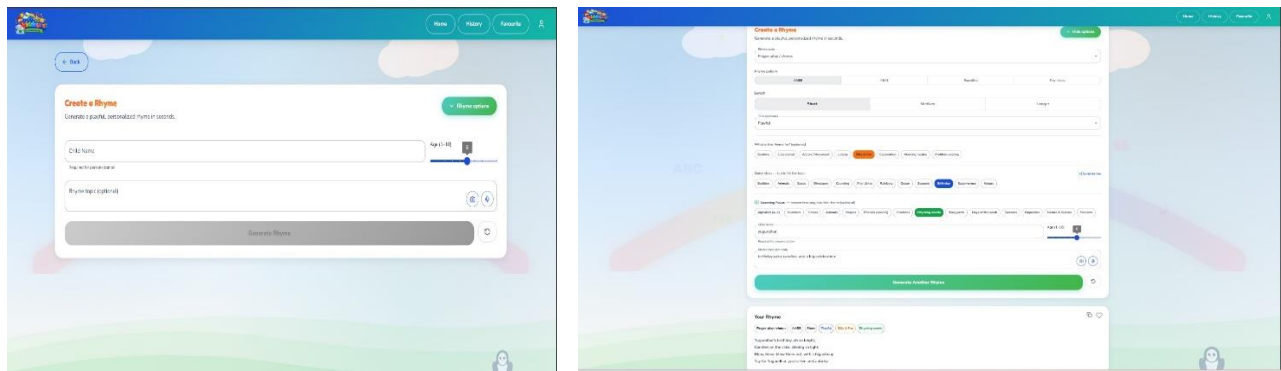


Figure 8: UI/UX Design – Create Rhyme Page (/home/create-rhyme) showing full layout from child name and age slider through Quick Ideas, Rhyme Options panel, Generate button, and rhyme output card

The CreateRhymePage (/home/create-rhyme) is built as a vertically stacked layout optimized for the content creation flow. From top to bottom: app bar with back navigation, page title with subtitle, child name input field, age slider (MUI Slider with tooltip), topic text area with voice mic and camera icons in the bottom-right corner, Quick Ideas chips row (12 options in a horizontal scroll row), Rhyme Options collapsible panel (teal accent, contains Style/Pattern/Length/Tone/Purpose/LearningFocus controls), Active Options summary strip (visible when panel is collapsed), Surprise Me randomizer link, Generate Rhyme button (full-width, teal gradient), Reset button (icon-only), and rhyme output card (displays generated rhyme with applied settings chips, copy button, and heart favourites button).

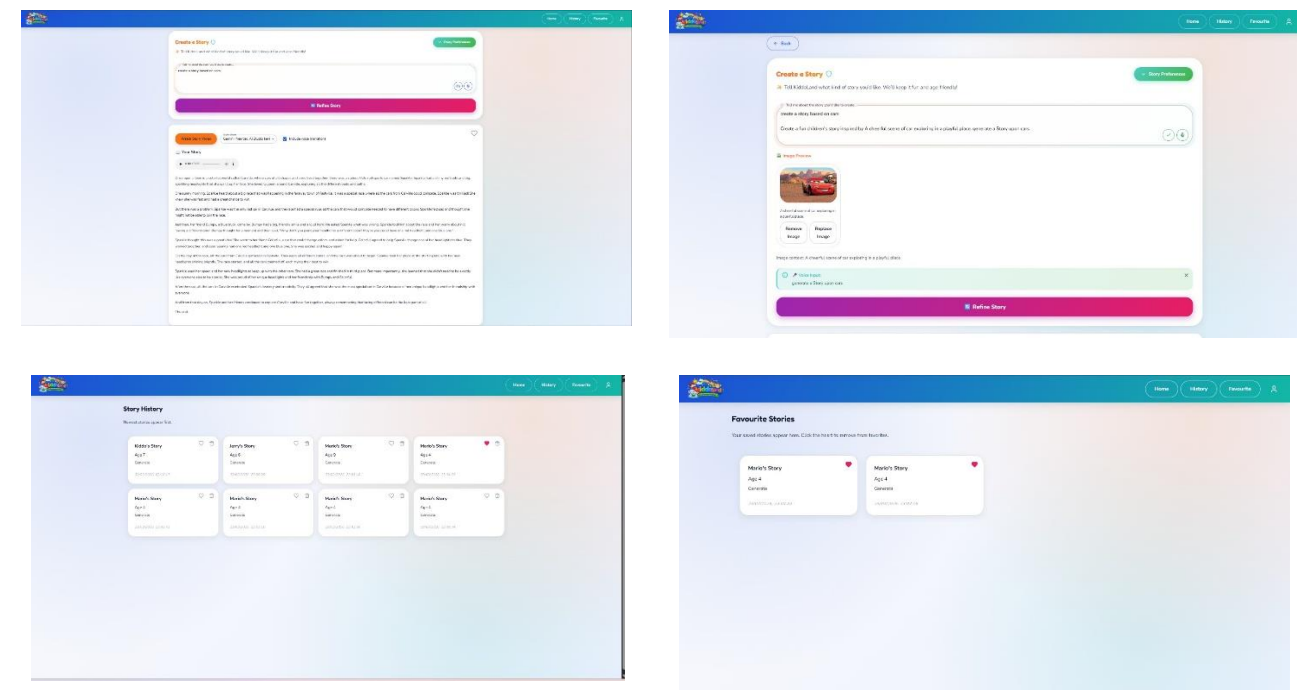


Figure 9: UI/UX Design – Create Story Page showing TTS toggle in prompt area, Advanced Preferences panel, Generate Story button, and StoryPreviewPanel with embedded audio player

The CreateStoryUnifiedPage (/home/create-story) features the unified input architecture combining all input modalities. The text prompt area has three icon buttons in the bottom-right corner: TTS toggle (Volume2/VolumeX, shown in blue when TTS is enabled), camera button (ImageUploadButton), and microphone button (VoiceInputButton, shown with red pulse when recording). The Advanced Preferences panel (collapsible, navy accent) contains age band, interests chips, tone, story type, length, mood, language, and learning goal. The Generate Story button (full-width, deep navy gradient) and Reset button (icon-only, red accent) are positioned below the preferences panel. When a story is generated, the StoryPreviewPanel renders below showing: the story title, the formatted Markdown story text, an HTML audio player (visible only when TTS was requested), a Copy to Clipboard button, a Refine Story button, and a Save to Favourites heart button.

Appendix G Deployment Strategy

Backend Deployment

Step	Command / Action	Notes
1 Clone	git clone https://github.com/premporiya/KiddoLand-Platform-API	Clones the backend repository
2 Virtual env	python -m venv venv	Creates isolated Python environment
3 Activate	venv\Scripts\activate (Windows) or source venv/bin/activate (Mac/Linux)	Must be activated before installing packages

Step	Command / Action	Notes
4 Install deps	<code>pip install -r requirements.txt</code>	Installs FastAPI, Pydantic, pymongo, sentence-transformers, httpx, gTTS, etc.
5 Create .env	Create .env file in project root with all required variables	See table below for required variables
6 Start server	<code>uvicorn main:app --reload --host 0.0.0.0 --port 8000</code>	Use --reload for development; remove for production
7 Verify	Open <code>http://localhost:8000/docs</code> in browser	Swagger UI should display all 18 endpoints

Environment Variable	Description	Example / Default
HUGGINGFACE_API_TOKEN	Hugging Face API token (free tier available at huggingface.co)	hf_XXXXXXXXXXXXXXXXXXXX
HUGGINGFACE_API_URL	Inference API base URL	<code>https://api-inference.huggingface.co/models/</code>
HUGGINGFACE_MODEL_ID	LLM model for generation	mistralai/Mixtral-8x7B-Instruct-v0.1
HUGGINGFACE_TTS_MODEL	TTS model ID	hexgrad/Kokoro-82M
MONGODB_URI	MongoDB connection string (optional)	<code>mongodb://localhost:27017</code>
MONGODB_DB_NAME	Database name	kiddoland
KIDDOLAND_AUTH_SECRET	64+ character random string for HMAC signing	(generate with: <code>openssl rand -hex 32</code>)
KIDDOLAND_AUTH_TTL_SECONDS	Token expiry in seconds	3600 (1 hour)

Frontend Deployment

Step	Command / Action	Notes
1 Clone	<code>git clone https://github.com/premporiya/KiddoLand-Platform-UI</code>	Clones frontend repository
2 Install	<code>npm install</code>	Installs React, TypeScript, MUI, lucide-react, and all dependencies
3 Create .env	<code>VITE_API_BASE_URL=http://localhost:8000</code>	Must match the backend server URL

Step	Command / Action	Notes
4 Development	npm run dev	Starts at http://localhost:5173 with hot module replacement
5 Production	npm run build	Creates optimized dist/ folder (~145KB gzipped)
6 Deploy	Upload dist/ to Netlify or Vercel	Static hosting; set VITE_API_BASE_URL to production backend URL

Demo Credentials (No MongoDB Required)

Account Type	Email	Password	Mode / Role
Parent (Home Mode)	parent@kiddoland.local	Parent123!	Home / Parent
Teacher (Institution Mode)	teacher@kiddoland.local	Teacher123!	Institution / Teacher
Admin (Institution Mode)	admin@kiddoland.local	Admin123!	Institution / Admin

Appendix H High-Level MLOps Description

KiddoLand uses inference-only MLOps no custom model training was performed. Our MLOps model is designed around three principles: environment-driven model selection, automatic fallback chains, and local semantic inference.

MLOps Concern	KiddoLand Approach
Model serving	All LLM and TTS inference handled by the Hugging Face Inference API (serverless, managed). No self-hosted GPU server required.
Model selection	Configured entirely via environment variables (HUGGINGFACE_MODEL_ID, HUGGINGFACE_TTS_MODEL, image generation model). Changing the LLM requires only an environment variable update no code change.
Fallback chains	TTS: Hugging Face → gTTS (automatic, transparent). Image generation: SDXL-Lightning → FLUX.1-schnell → Gemini API (user-selectable).
Local semantic model	all-MiniLM-L6-v2 loads at application startup from sentence-transformers library. Runs on CPU locally. Deterministic, no API key, no cost per call. Used only for book recommendation ranking.
Monitoring	Python logging module writes warnings for all HF API failures, token validation failures, and safety filter triggers. Accessible in uvicorn console output.
Model drift	Not applicable no custom model was trained. Output quality depends entirely on the hosted Hugging Face model, maintained by Hugging Face.
Prompt versioning	Age guidance templates in <code>_get_age_guidance()</code> are version-controlled in source code. Changes to prompts require code changes and pull request review.

Appendix I List of Acronyms

Acronym	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BLEU	Bilingual Evaluation Understudy
CBO	Coupling Between Objects (software metric)
COPPA	Children's Online Privacy Protection Act
CORS	Cross-Origin Resource Sharing
CVC	Consonant-Vowel-Consonant (simple word structure in early literacy)
DIT	Depth of Inheritance Tree (software metric)
EDA	Exploratory Data Analysis
GDPR	General Data Protection Regulation (European Union)
HF	Hugging Face
HMAC	Hash-based Message Authentication Code
HTTP	HyperText Transfer Protocol
LCOM	Lack of Cohesion in Methods (software metric)
LLM	Large Language Model
MCQ	Multiple Choice Question
MUI	Material User Interface (Material UI component library)
NLP	Natural Language Processing
PBKDF2	Password-Based Key Derivation Function 2
PII	Personally Identifiable Information
PIPEDA	Personal Information Protection and Electronic Documents Act (Canada)
RFC	Response For a Class (software metric)
RLHF	Reinforcement Learning from Human Feedback
REST	Representational State Transfer
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
SHA	Secure Hash Algorithm
STS	Semantic Textual Similarity
TTS	Text-to-Speech
UAT	User Acceptance Testing

Acronym	Full Form
UI	User Interface
UX	User Experience
WCAG	Web Content Accessibility Guidelines
WMC	Weighted Methods Per Class (software metric)

Appendix I Glossary of Terms

Term	Definition
Age Band	A grouped age range (e.g., Ages 3–4, Ages 5–6) used to categorize a child's developmental stage and apply appropriate vocabulary complexity, sentence length, and narrative structure in generated content.
Age Guidance	A system message string injected into every LLM call by <code>_get_age_guidance(age)</code> that instructs the model on the vocabulary level, sentence complexity, and structural expectations appropriate for the specified age. The single most important prompt engineering decision in KiddoLand.
<code>buildRhymePrompt()</code>	A TypeScript function in <code>rhymePrompt.ts</code> that converts up to 7 simultaneously selected rhyme parameters into a single, coherent natural-language instruction string sent to the backend.
Constraint-first pipeline	KiddoLand's core AI safety architecture: all safety and age-appropriateness constraints are enforced before sending any prompt to the LLM, and again on the LLM's output – never relying on the model's fine-tuning alone.
Cosine similarity	A measure of similarity between two vector embeddings computed as their dot product (when both are L2-normalized). Values range from 0 (no similarity) to 1 (identical). Used in KiddoLand for ranking book recommendations.
Domain boost	An additional +0.1 score increment applied in the semantic book ranking pipeline when a book's subject text strongly matches the topic of the search query (e.g., the word 'space' appears in both the query and the book's subject list).
Dual-layer safety filter	Content safety validation applied at two independent checkpoints: once on user input (pre-generation) and once on LLM output (post-generation). If either check fails, safe fallback behaviour is triggered.
gTTS	Google Text-to-Speech – a Python library used as a fallback TTS provider when the Hugging Face TTS API is unavailable or returns a non-200 response.
Home Mode	KiddoLand's account-based operating mode for parents and guardians, supporting personalized content with child name/age, story history, favorites, TTS narration, and full preference customization.
Hugging Face Inference API	A hosted API service by Hugging Face that runs inference on large AI models without requiring users to manage GPU servers. KiddoLand uses it for LLM text generation, TTS audio, and experimental image generation.
Institution Mode	KiddoLand's privacy-first operating mode for schools, libraries, and classrooms. All sessions are anonymous; no child names or personal identifiers are stored. All backend records use 'Class' as a placeholder.
PBKDF2	Password-Based Key Derivation Function 2 – a standard algorithm for securely hashing passwords. KiddoLand uses 100,000 iterations of SHA-256 with a 16-byte random salt per user.

Term	Definition
Prompt engineering	The practice of constructing structured natural-language instructions that guide an LLM to produce a specific type of output. In KiddoLand, the system message (age guidance) and user message (structured prompt with all parameters) are the two key engineered elements.
Regex (Regular Expression)	A pattern-matching language used in <code>safety_filter.py</code> to detect prohibited keywords. Chosen for its 100% deterministic rejection of defined patterns essential for child-facing safety applications.
Semantic embedding	A fixed-size numerical vector representation of text produced by a sentence-transformer model. Semantically similar texts produce vectors with high cosine similarity, enabling meaning-based ranking rather than keyword matching.
sentence-transformers	A Python library providing pre-trained models (including all-MiniLM-L6-v2) for producing normalized sentence embeddings. Used in KiddoLand for book recommendation ranking.
Session Profile	The structured collection of user preferences (age, interests, tone, learning goal, rhyme style, etc.) assembled from form inputs and used to construct the personalized LLM prompt.
Surprise Me	A feature on the Rhyme Creation page that randomizes all seven rhyme parameters simultaneously with a single click, enabling rapid exploration of different rhyme styles.
TTS (Text-to-Speech)	Technology that converts written text to spoken audio. KiddoLand uses the Hugging Face TTS API (hexgrad/Kokoro-82M) as the primary provider with gTTS as a fallback.
Unified input interface	KiddoLand's story and rhyme creation design pattern: text input, voice input (microphone), and image input (camera) are all accessible from a single textarea with overlaid icon buttons no separate pages or steps required.
Web Speech API	A browser-native JavaScript API providing speech recognition (<code>SpeechRecognition</code> / <code>webkitSpeechRecognition</code>) for real-time voice-to-text transcription. Used in KiddoLand with <code>continuous: true</code> and <code>interimResults: true</code> for real-time appending to the prompt field.